Universitatea Transilvania din Brașov
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ

Programul de studiu:
Informatică

Lucrare de licență

Sistem de procesare și vizualizare a imaginilor cu suport pentru filtre avansate și procesare paralelă

Autor: Horovei Dumitru
Coordonator științific: Prof. Dr. Deaconu Adrian-Marius

Brașov – 2025

Cuprins

Capitolul 1 4
1. Introducere 4
1.1. Context general 4
1.2. Scopul aplicației 4
1.3. Motivatia aplicației 4
1.4. Comparație cu alte aplicații existente 4
1.5. Tehnologii folosite 5
1.6. Structura lucrării 5
Capitolul 2 6
2. Noțiuni teoretice 6
2.1. Spații de culoare 6
2.1.1. Spațiul de culoare RGB 7
2.1.2. Spațiul de culoare HSV 8
2.1.3. Spațiul de culoare CMYK 9
2.2. Filtre de imagine implementate 10
2.2.1. Conversia la nivele de gri (Grayscale) 10
2.2.2. Inversarea culorilor (Invert) 11
2.2.3. Răsturnare orizontală și verticală (Flip) 12
2.2.4. Filtru de estompare (Blur) 13
2.2.5. Filtrul de accentuare a claritității (Sharpen) 14
2.2.6. Efect mozaic (Mosaic) 17
2.2.7. Filtru Sepia 19
2.2.8. Efect pictură în ulei (Oil Paint) 20
2.2.9. Efect glitch 22
2.2.10. Efect caleidoscop (Kaleidoscope) 24

2.2.11. Filtrul Sobel 25
2.2.12. Filtrul Canny 29
2.2.13. Compararea filtrelor Sobel și Canny 33
Capitolul 3 35
3. Tehnologii folosite 35
3.1. C++ și OpenMP 35
3.2. .NET 8 (C# și REST API) 36
3.3. Angular 19 (Frontend web) 38
3.4. MongoDB 39
3.5. Dropbox (stocare fișiere) 40
3.6. Doxygen 41
3.7. Docker 42
3.8. Github Actions (CI/CD) 43
3.9. Limbaje de programare utilizate: C++, C# și TypeScript 44
3.10. Medii de dezvoltare: Visual Studio și Visual Studio Code 46
Capitolul 4 48
4. Prezentarea aplicației: facilități și mod de utilizare 48
4.1. Descriere generală 48
4.2. Facilități principale 48
4.3. Fluxul de utilizare 49
4.3.1. Pagina principală 49
4.3.2. Editarea imaginilor 49
4.3.3. Galerie de imagini 50
4.3.4. Vizualizarea detaliată (Dialog) 51
4.3.5. Vizualizarea arborescentă 52
Capitolul 5 53
5. Concluzii și rezultate 53

5.1. Direcții de extindere 55

6. Bibliografie 58

Capitolul 1

1. Introducere

1.1. Context general

În contextul actual, procesarea digitală a imaginilor reprezintă un domeniu vital atât pentru aplicațiile profesionale, cum ar fi cele medicale sau din industria grafică, cât și pentru aplicații de larg consum precum editarea fotografiilor personale sau crearea de conținut artistic și multimedia. Aplicația dezvoltată în cadrul acestei lucrări propune o soluție avansată pentru procesarea imaginilor, care combină performanța și flexibilitatea prin utilizarea unei arhitecturi stratificate și a unor tehnologii moderne și optimizate.

1.2. Scopul aplicației

Scopul principal al aplicației este de a oferi utilizatorilor o soluție performantă și accesibilă pentru procesarea și editarea rapidă a imaginilor, integrând o gamă largă de filtre și efecte speciale într-o interfață ușor de folosit.

1.3. Motivatia aplicației

Motivația alegerii acestei teme vine din nevoia unei aplicații performante, prietenoase și eficiente pentru editarea rapidă și facilă a imaginilor, care sa depășească unele limitări existente în soluțiile actuale, cum ar fi timpi mari de procesare, suport limitat pentru diferite platforme și dificultăți în utilizare.

1.4. Comparație cu alte aplicații existente

Există pe piață o serie de aplicații populare pentru editarea imaginilor, cum ar fi Adobe Photoshop, GIMP sau Pixlr, fiecare având propriile avantaje și dezavantaje. În comparație cu aceste aplicații, soluția dezvoltată se remarcă prin implementarea unui motor de procesare optimizat în limbajul C++, folosind paralelizarea cu OpenMP, integrat într-un serviciu backend .NET robust și un frontend Angular modern, facilitând o experiență de utilizare performantă și fluidă. De asemenea, aplicația oferă suport avansat pentru filtre artistice și efecte speciale precum: glitch, oil paint și kaleiidoscope, integrate într-o interfață web responsivă și usor de utilizat.

Inițial, proiectul a folosit Google Drive pentru stocarea imaginilor încărcate de utilizatori. Cu toate acestea, în urma unor probleme identificate în timpul dezvoltării, s-a optat pentru schimbarea serviciului de stocare către Dropbox, ceea ce a îmbunătățit semnificativ stabilitatea și fiabilitatea aplicației.

1.5. Tehnologii folosite

Tehnologiile folosite în acest proiect sunt alese strategic pentru a asigura performanța, scalabilitatea și compatibilitatea multiplatformă:

- **C++** și **OpenMP** pentru biblioteca performantă și optimizată de procesare a imaginilor;
- **.NET 8** pentru implementarea API-ului REST robust și scalabil, cu integrare **SignalR** pentru actualizări în timp real;
- **Angular 19** pentru interfața web moderna, reactivă și intuitivă pentru utilizatori;
- **MongoDB** și **Dropbox** pentru stocarea fiabilă și eficientă a imaginilor și datelor aplicației;
- **Docker** pentru containerizarea aplicației backend, asigurând suport multiplatforma pentru Windows și Linux;
- **Github Actions** pentru implementarea unui pipeline **CI/CD** complet, cu teste automate și integrare continuă pentru toate componentele proiectului;
- **Doxygen** pentru generarea automată a documentației codului sursă.

1.6. Structura lucrării

În capitolele următoare ale acestei lucrări vom detalia fundamentul teoretic al fitrelor utilizate, tehnologiile alese și motivația utilizării acestora, limbajele și mediile de programare implicate, vom prezenta detaliat funcționalitățile aplicației cu exemple practice, iar în final vom prezenta concluziile obținute, limitările întâmpinate și posibile direcții viitoare de dezvoltare.

Capitolul 2

2. Noțiuni teoretice

Procesarea imaginilor digitale reprezintă ansamblul tehnicilor și algoritmilor prin care imagini în format digiital sunt analizate și modificate pentru a îmbunătăți calitatea acestora, a extrage informații utile sau a le face adecvate unui anumit scop. Domeniul are aplicabilitate extinsă, de la fotografie și design grafic până la imagistică medicală și sisteme de supraveghere, jucând un rol vital în numeroare industrii moderne. În esență, un sistem de procesare a imaginilor prelucrează valorile pixelilor (elementele de bază ale imaginii) folosind diverse filtre și transformări matematice, pentru a produce o imagine de ieșire cu anumite caracteristici modificate față de imaginea de intrare.

În continuare, vom prezenta conceptele fundamentale legate de procesarea imaginilor digitale: spațiile de culoare și transformările între ele, precum și o serie de filtre de imagine implementate în cadrul aplicației proiectului. Pentru fiecare filtru sunt oferite explicații teoretice, formule matematice relevante, algoritmul de implementare (descris în cuvinte sau pseudocod) și un fragment din codul C++ utilizat. De asemenea, se va discuta utilizarea bibliotecii *OpenMP* pentru optimizarea execuției acestor filtre în paralel.

2.1. Spații de culoare

Un *spațiu de culoare* reprezintă o modalitate matematică de a modela și specifica culorile prin seturi de valori numerice. Alegerea spațiului de culoare este esențială în procesarea imaginilor, întrucât definește modul în care sunt codificate culorile ți intervalele de valori disponibile pentru fiecare componentă de culoare. Trei spații de culoare foarte răspândite sunt modelul aditiv *RGB* (Red, Green, Blue), modelul perceptual *HSV* (Hue, Saturation, Value) și modelul substractiv *CMYK* (Cyan, Magenta, Yellow, Key/Black) utilizat predominant în tipografiie. Vom descrie în continuare aceste spații de culoare, împreună cu formulele de conversie relevante.

2.1.1. Spațiul de culoare RGB

Modelul RGB (Roșu – Verde – Albastru) definește culorile pe baza intensităților combinate ale celor trei *culori primare aditive:* roșu (R), verde (G) și albastru (B). Modelul RGB este un model de culoare aditiv în care luminile primare roșu, verde și albastru sunt adăugate în diferite proporții pentru a reproduce o gamă largă de culori. De exemplu, valoarea maximă a tuturor celor trei componente produce alb, în timp ce valoarea minima (zero) produce negru. În practiica uzuală. Fiecare componentă este codificată pe 8 biți (interval 0 -255), ceea ce permite reprezentarea a $256^3 \approx 16.7$ milioane de culori distincte.

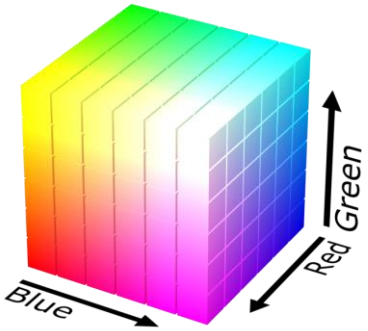

Fig. 2.1.: Cubul RGB

Spațiul RGB poate fi imaginat geometric ca un *cub al culorilor*, unde fiecare axă corespunde uneia dintre componente. Orice culoare este un punct în acest spațiu 3D, de coordonate (R, G, B).

Conversia între spațiul RGB și alte spații de culoare este un pas frecvent în preprocesarea imaginilor. De exemplu, conversia unei imagini RGB în *grayscale* (tonuri de gri) se poate realiza printr-o medie ponderată a componentelor:
$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$, care ține cont de sensibilitatea inegală a ochiului uman la culorile primare. Acești coeficienți (conform standardului ITU BT.601) dau o luminozitate percepută corectă imaginii gri rezultate. Totuși, un spațiu de culoare dedicat percepției, precum HSV, poate fi uneori preferat pentru anumite prelucrări, deoarece separă nuanța de intensitate.

2.1.2. Spațiul de culoare HSV

Modelul HSV (Hue, Saturation, Value), numit și *HSB* (Hue, Saturation, Brightness), reprezintă culorile într-o manieră mai apropiată de percepția umană, separând informația despre *nuanță* (culoarea de bază) de *saturație* (puritatea culorii) și *valoare* (strălucirea sau luminozitatea).

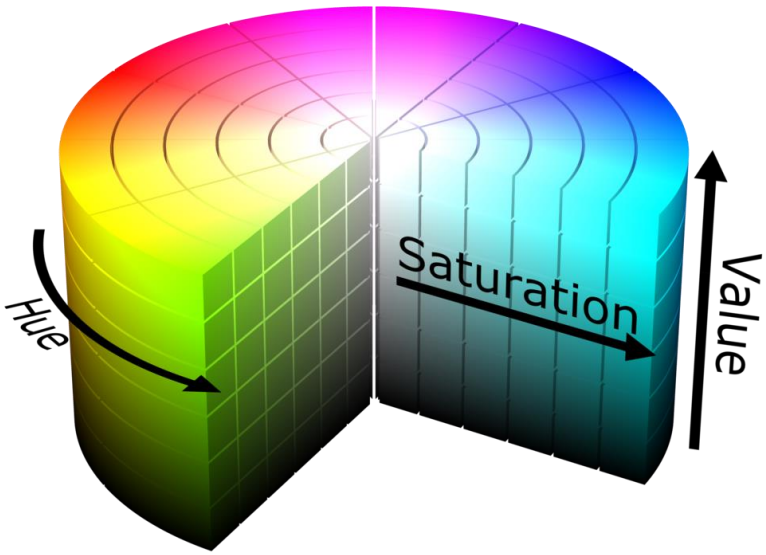

Fig. 2.2.: Cilindrul HSV

Spațiul HSV poate fi reprezentat geometric ca un *cilindru al culorilor*, unde unghiul de rotație reprezintă nuanța (H, în grade de la 0° la 360°), distanța radială față de axă reprezintă saturația (S între 0% și 100%), iar înălțimea pe axa vertiicală reprezintă valoarea (V între 0% - negru – și 100% - maximul de lumină). Astfel, HSV decuplează aspectul cromatic de luminozitate, ceea ce facilitează anumite prelucrări precum schimbarea nuanței unei imagini fără a-i altera luminozitatea.

Conversia din spațiul RGB în HSV se realizează printr-o serie de pași care implică identificarea componentelor minimă și maximă dintre R, G și B. Fie valorile normalizate (între 0 și 1) R', G', B'. Calculăm mai întâi valorile extremă:

$$C_{max} = \max(R', G', B'), C_{min} = \min(R', G', B'), \Delta = C_{max} - C_{min}$$

Nuanța (H) se calculează în funcție de ce componentă este maximă:

$$H = \begin{cases} 0, & \Delta = 0 \\ \left(60^\circ \cdot \frac{G' - B'}{\Delta} + 360^\circ\right) mod\ 360^\circ, C_{max} = R' \\ 60^\circ \cdot \frac{B' - R'}{\Delta} + 120^\circ, C_{max} = G' \\ 60^\circ \cdot \frac{B' - R'}{\Delta} + 240^\circ, C_{max} = B' \end{cases}$$

Saturația se definește ca raportul dintre diferența Δ și valoarea maximă:

$$S = \begin{cases} 0, C_{max} = 0 \\ \frac{\Delta}{C_{max}}, C_{max} \neq 0 \end{cases}$$

Valoarea (V) corespunde direct componentei cele mai intense:

$$V = C_{max}$$

Modelul HSV este foarte util în aplicații de procesare unde se dorește manipularea separată a nuanței față de luminozitate – de exemplu, pentru filtre de schimbare a culorii (hue shifting) sau pentru detecția obiectelor pe baza culorii indiferent de iluminare.

2.1.3. Spațiul de culoare CMYK

Modelul CMYK (Cyan, Magenta, Yellow, blacK) este un model de culoare *substractiv*, folosit preponderent în domeniul tipăririi color. În loc să se combine lumini colorate ca în RGB, modelul CMYK pornește de la hârtia albă (care reflectă toate culorile) și *subtractează* componente de culoare folosind cerneluri transparente cyan (C), magenta (M), galben (Y) și negru (K) pentru a absorbi anumite lungimi de undă ale luminii.

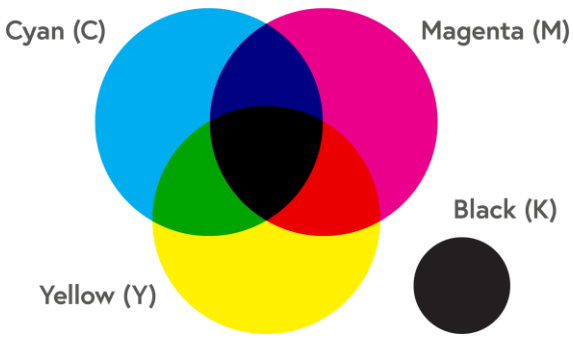

Fig. 2.3.: Modelul CMYK

Combinațiile acestor pigmenți pot reproduce un spectru larg de culori percepute atunci când lumina albă este reflectată de pe hârtie. Negrul (K, denumit *key*) este folosit suplimentar pentru a intensifica densitatea și contrastul, deoarece amestecul cernelurilor C, M, Y nu produce un negru pur și bogat.

Conversia din RGB în CMYK implică mai întâi normalizarea valorilor R, G, B în intervalul [0, 1] ca R', G', B'. Se calculează componenta de *cheie* (K) care reprezintă cantitatea de negru, ca: $K = 1 - \max(R', G', B')$. Apoi componentele C, M, Y rezultă din diferența față de această valoare, raportată la intensitatea rămasă (1 – K):

$C = \frac{1 - R' - K}{1 - K}, M = \frac{1 - G' - K}{1 - K}, C = \frac{1 - B' - K}{1 - K}.$

Dacă K = 1 (culoare neagră absolută), atunci prin convenție se setează $C = M = Y = 0$. Valorile se pot apoi scala în procente 0 – 100% pentru fiecare canal de cerneală.

2.2. Filtre de imagine implementate

În cadrul proiectului au fost implementate mai multe filtre de procesare a imaginii, fiecare realizând o anumită transformare asupra pixelilor imaginii. Vom descrie în continuare, pentru fiecare filtru, prinicpiul teoretic de funcționare, ecuațiile sau operațiile implicate, precum și detalii despre algoritmul de implementare în C++ (inclusiv optimizări cu OpenMP acolo unde este cazul).

2.2.1. Conversia la nivele de gri (Grayscale)

Filtrul Grayscale transformă o imagine color într-o imagine în tonuri de gri, eliminând informația de culoare și păstrând doar intensitatea luminoasă. Teoretic, conversia se poate realiza fie printr-o medie simplă a canalelor R, G, B ($Y = \frac{R+G+B}{3}$), fie printr-o medie ponderată. Formula standard de luminozitate folosește ponderi bazate pe sensibilitatea ochiului: $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$, acordând importanță mai mare canalului verde și mai mică celui albastru. Rezultatul este o singură valoare de intensitate (0 = negru, 255 = alb) pentru fiecare pixel.

Algoritmul de conversie este direct: se parcurg toți pixelii imaginii și pentru fiecare se calculează componenta gri aplicând formula de mai sus. Acest lucru se poate implementa eficient parcurgând vectorul de date al imaginii și înlocuind tripletele (R, G, B) cu valoarea calculată Y. Un fragment relevant din implementarea C++ arată cum se realizează parcurgerea în paralel a pixelilor (folosind OpenMP) și conversia fiecărui pixel la gri:

```
#pragma omp parallel for
    for (int i = 0; i < size; i += channels)
    {
        unsigned char r = inputImage[i];
        unsigned char g = inputImage[i + 1];
        unsigned char b = inputImage[i + 2];
        unsigned char gray = static_cast<unsigned char>(r * 0.3f + g * 0.59f + b * 0.11f);
        outputImage[i] = gray;
        if (channels > 1 && i + 1 < size) outputImage[i + 1] = gray;
        if (channels > 2 && i + 2 < size) outputImage[i + 2] = gray;
    }
```

Fig. 2.4.: Conversia la tonuri de gri

În urma acestui filtru, imaginea devine monocromatică, reținând structura și detaliile luminoare, dar pierzând informația despre culoare. În exemplul din Figura de mai sus, fotografia originală (stânga) este convertită în versiunea grayscale (dreapta), păstrând claritatea contururilor și a luminilor/umbrelor, dar fără elementele cromatice distinctive.

2.2.2. Inversarea culorilor (Invert)

Filtrul Negative (Invert) transformă o imagine în *negativul* ei fotografic, prin inversarea fiecărei componente de culoare. Pentru imagini codificate pe 8 biți per canal, operația matematică este foarte simplă: pentru un pixel cu valoarea v (între 0 și 255), valoarea inversată devine 255 – v. Astfel, alb (255) devine negru (0), negrul devine alb, iar culorile se transformă în complementarele lor (de exemplu, roșu devine cyan, verde devine magenta, albastru devine galben).

Acest filtru se aplică independent pe fiecare valoare de canal R, G, B a imaginii. În loc să se aplice o scădere explicită din 255 pentru fiecare canal, implementarea utilizeaza operația XOR cu masca binara 0xFF (adică 255 în baza 10), care este echivalentă cu inversarea fiecărui bit al valorii. Această metodă oferă o inversare rapidă și eficientă a valorii pixelilor. Fragmentul de cod C++ relevant care ilustrează această abordare:

```
1. #pragma omp parallel for
2.     for (int i = 0; i < size; i++)
3.     {
4.         outputImage[i] = inputImage[i] ^ 0xFF;
5.     }
```

Fig. 2.5.: Inversarea culorilor

Efectul vizual al filtrului este unul puternic: zonele luminoase devin întunecate și viceversa, iar culorile sunt complementare față de original.

2.2.3. Răsturnare orizontală și verticală (Flip)

Filtrul de Flip (oglindire) întoarce imaginea fie orizontal (stânga – dreapta), fie vertical (sus – jos), producând o imagine oglindită față de original. Acest tip de procesare nu modifică valorile de culoare ale pixelilor, ci doar le *rearanjează pozițiile* în raster. În cazul flip-ului orizontal, coloana din stânga devine coloana din dreapta și viceversa – practic imaginea este reflectată față de o axă verticală care trece prin mijlocul său. Similar, flip-ul vertical reflectă imaginea în jurul unei axe orizontale mediane (sus devine jos și invers.

Algoritmul pentru flip orizontal: pentru fiecare pixel de la coordonatele (x,y), se va copia valoarea lui în poziția (width -1 – x, y) din imaginea de ieșire. În implementarea C++, acest lucru se poate realiza eficient parcurgând liniile imaginii și copiindu-le inversate. De exemplu, pentru flip vertical, se pot inversa ordinea liniilor întregi (swap între linia i și linia height – 1 – i). Codul de mai jos arată folosirea OpenMP pentru a paraleliza copierea liniilor unei imagini în vederea răsturnării verticale:

```
 1. #pragma omp parallel for
 2.     for (int y = 0; y < height; y++)
 3.     {
 4.         int srcRowOffset = y * rowSize;
 5.         int dstRowOffset = (height - y - 1) * rowSize;
 6.         std::copy(inputImage + srcRowOffset, inputImage + srcRowOffset + rowSize,
outputImage + dstRowOffset);
 7.     }
```

Prin această abordare se copiază toți pixelii liniei y din imaginea sursă în linia simetrică față de centru în imaginea destinație, realizând efectul de oglindire verticală.

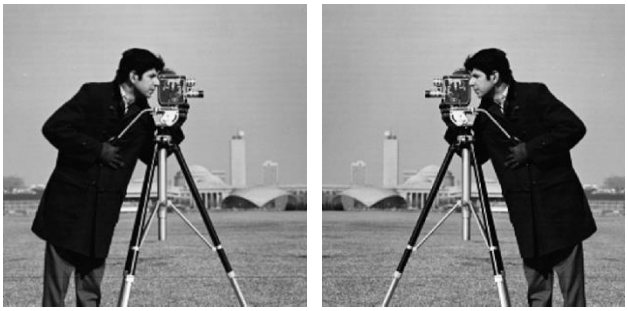

Fig. 2.6.: Răsturnare orizontală

Pentru oglindirea orizontală, un procedeu analog este aplicat pe coloane (sau, alternativ, tot pe linii dar accesând pixelii în ordine inversă). Figura de mai sus exemplifică un flip orizontal: imaginea originală (stânga) și cea oglindită (dreapta). Se observă că structura generală rămâne aceeași, dar elementele sunt inversate lateral.

2.2.4. Filtru de estompare (Blur)

Filtrul Blur (estompare) realizează *netezirea* imaginii prin reducerea variațiilor bruște dintre pixeli, rezultând o imagine mai încețoșată. Acest efect se obține prin înlocuirea fiecărui pixel cu o medie a pixelilor din vecinătatea sa. Prin mediere, detaliile fine (cum ar fi zgomotul sau marginile foarte ascuțite) sunt atenuate. Un caz simplu este filtrul *mean* (medie simplă) cu o mască pătrată uniformă. De exemplu, un bluc cu masca 3×3 va lua pentru fiecare pixel media celor 9 pixeli (el însuși și cei 8 vecini adiacenți), înlocuind pixelul central cu această medie. Pentru o mască mai mare (ex: 5×5, 7×7), efectul de estompare este mai pronunțat.

Matematic, pentru un blur cu fereastră pătrată de latură $m = 2r + 1$ (raza r pixeli în jurul punctului central), valoarea filtrată într-un pixel (x, y) devine:
$I_{blur}(x, y) = \frac{1}{m^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} I_{in}(x + i, y + j)$, unde I_{in} este imaginea inițială iar suma se calculează pe toți vecinii din fereastră (cu tratament special la margini, de exemplu extindere prin oglindire sau replicare).

Implementarea naivă a acestei convoluții are complexitate $O(N \cdot m^2)$ (unde N este numărul de pixeli), ceea ce poate fi costisitor pentru ferestre mari. Pentru optimizare, se poate folosi tehnica *imaginii integrale* (integral image) pentru a calcula mai rapid sumele pe blocuri. Codul din proiect folosește această metodă: mai intâi calculează imaginea integrală a fiecărui canal, apoi determină suma valorilor dintr-o fereastră dată în O(1) folosind diferențe de zone integrate. Fragmentul de mai jos

ilustrează utilizarea imaginii integrale pentru a obține rapid suma pixelilor dintr-un *tile* (bloc) înconjurător, urmată de calculul mediei:

```
 1. // Determinare coordonate bloc (x0,y0) - (x1,y1) de blur
 2. int x0 = std::max(x - halfKernel, 0);
 3. int y0 = std::max(y - halfKernel, 0);
 4. int x1 = std::min(x + halfKernel, width - 1);
 5. int y1 = std::min(y + halfKernel, height - 1);
 6. int area = (x1 - x0 + 1) * (y1 - y0 + 1);
 7. // Suma pe bloc folosind integrala (A, B, C, D sunt colțurile blocului în integrală)
 8. uint64_t sum = integral[idxD];
 9. if (idxB != -1) sum -= integral[idxB];
10. if (idxC != -1) sum -= integral[idxC];
11. if (idxA != -1) sum += integral[idxA];
12. // Valoarea medie pe bloc
13. outputImage[idxOut] = static_cast<unsigned char>(sum / area);
```

Acest cod (simplificat pentru un canal) calculează sumele pe zona pătrată și apoi împarte la numărul de pixeli, obținând valoarea medie ce va fi atribuită pixelului *output*. Rezultatul final este o imagine în care detaliile fine au fost estompate.

Fig. 2.7.: Filtru de estompare

În exemplul prezentat, imaginea din stânga reprezintă fotografia originală, unde detaliile feței ale cameramanului și ale fundalului sunt clar definite. Imaginea din dreapta ilustrează rezultatul aplicării filtrului Blur, unde contururile devin vizibil mai moi, iar detaliile mai fine, cum ar fi texturile hainelor sau structurile din fundal, sunt estompate. Filtrul de estompare reduce astfel zgomotul vizual și varianța rapidă a pixelilor, conferind imaginii un aspect mai neted, dar și o ușoară pierdere de claritate. Acest tip de filtru este util pentru reducerea zgomotului sau generarea de efecte artistice.

2.2.5. Filtrul de accentuare a claritățiii (Sharpen)

Filtrul Sharpen este utilizat în procesarea imaginilor pentru a *accentua detaliile* și ia face *marginile obiectelor mai clare*. Cu alte cuvinte, spre deosebire de filtrul Blur care netezește și estompează detaliile, filtrul Sharpen are efectul opus: *evidențiază*

diferențele de intensitate dintre pixeli adiacenți, sporind contrastul local. Aplicarea unui astfel de filtru conduce la creșterea calității percepute a imaginii, fiind adesea folosită în post-procesarea fotografiilor pentru a îmbunătăți claritatea sau pentru a compensa ușoare neclarități ale capturii originale. Totuși, este important de menționat că o accentuare excesivă poate amplifica și zgomotul sau artefactele din imagine, motiv pentru care filtrarea Sharpen se aplică de obicei ca ultim pas în fluxul de prelucrare și cu parametri controlați.

Principiu de funcționare. În termeni teoretici, un filtru de claritate este un *filtru trece-înalt* (high-pass) care pune în evidență componentele de frecvență înaltă ale imaginii (detaliile fine, muchiile) fără a elimina complet componentele de frecvență joasă (zonele uniforme). Acest lucru se poate realiza prin *diferențiere spațială* – practic calculând variațiile dintre un pixel și vecinii săi. O metodă directă de implementare este folosirea unui *kernel* (matrice de convoluție) care să amplifice pixelul central și să subtracte contribuțiile vecinilor. De exemplu, un kernel 3×3 tipic pentru sharpen este:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Acest kernel are valoarea centrală mai mare decât 1 și valori negative în jur (suma elementelor este 1), ceea ce înseamnă că la calculul convoluției pixelul original este accentuat, iar contribuțiile vecinilor sunt scăzute. Efectul rezultat este mărirea contrastului la marginile unde există diferențe semnificative între pixelul central și vecinii săi. O altă metodă populară de realizare a filtrului Sharpen este tehnica *Unsharp Mask* (mască de neclaritate). Paradoxal, denumirea de „unsharp" provine din fotografia tradițională și indică faptul că se folosește o versiune neclară a imaginii pentru a obține claritatea: se generează o copie blurată a imaginii originale, apoi aceasta este scăzută din original pentru a evidenția contururile, după care diferența rezultată (masca) este adăugată înapoi la imaginea originală. Formula generală a acestei metode este:

$I_{sharpen} = I_{original} + \lambda \cdot (I_{original} - I_{blur})$, unde I_{blur} este versiunea estompată (filtrată low-pass) a imaginii, iar λ este un factor de scalare (amount) ce controlează intensitatea clarității. Prin alegerea adecvată a acestui factor, se poate controla cât de puternic sunt amplificate detaliile. Procesul descris mai sus *mărește selectiv contrastul de-a lungul marginilor* detectate cu ajutorul măștii de neclaritate, lăsând în urmă o imagine cu aspect mai ascuțit și detalii mai bine definite.

Algoritm – pașii aplicării filtrului Sharpen: În implementare, filtrul de accentuare a claritățiii poate fi realizat urmând pașii de mai jos (corespunzători metodei Unsharp Mask):

1. *Blurarea imaginii originale:* se aplică un filtru de netezire (un filtru Gaussian sau media) asupra imaginii de intrare, obținându-se o versiune *blurată* a acesteia. Această imagine estompată va servi drept „mască de neclaritate".
2. *Calculul detaliilor (componenta de înaltă frecvență):* se calculează diferența pixel cu pixel între imaginea originală și cea blurată. Această diferență $D = I_{original} - I_{blur}$ evidențiază *detaliile și muchiile* din imagine (partea de frecvență înaltă care fusese estompată de filtrul de blur).
3. *Amplificarea diferențelor:* diferența calculată D este înmulțită cu un factor numeric amount (de obicei între 0.5 și 2.0, în funcție de cât de accentuat se dorește efectul). Aceasta produce $D' = \lambda \cdot D$, unde λ este factorul de accentuare.
4. *Recombinarea cu imaginea originală:* se adaugă componenta amplificată D' la imaginea originală: $I_{sharpen} = I_{original} + D'$. În acest fel, pixelii la nivelul marginilor (unde D era diferit de zero) vor fi intensificați, mărind contrastul local, în timp ce zonele uniforme (unde D ≈ 0) rămân practic neschimbate.
5. *Clamping (tăierea valorilor ieșite din interval):* dacă în urma adunării rezultă valori de intensitate în afara intervalului permis [0, 255] (în cazul imaginilor pe 8 biți), aceste sunt saturate la 0 sau 255 pentru a menține valorile valide ale pixelilor.

Implementarea practică a filtrului Sharpen folosește adesea paralelizare și operații pe fiecare pixel, dat fiind că pentru fiecare poziție în imagine calculul este independent (poate fi efectuat în paralel). Mai jos este prezentat un fragment de cod C++ care aplică un filtru Sharpen folosind tehnica Unsharp Mask, comentat pentru claritate:

```
 1. // Pasul 1: se creează o versiune blurată a imaginii de intrare
 2. std::vector<unsigned char> blurredImage(size);
 3. BlurFilter blurFilter;
 4. blurFilter.Apply(inputImage, blurredImage.data(), width, height, channels, nullptr);
 5.
 6. // Pasul 2: se aplică formula Unsharp Mask: output = original + amount * (original -
blurred)
 7. const float amount = 1.5f;  // factorul de accentuare a claritățiii (intensitate sharpen)
 8.
 9. #pragma omp parallel for
10. for (int i = 0; i < size; i++)
11. {
```

```
12.     // se calculează diferența dintre valoarea originală a pixelului și cea blurată
13.     int diff = static_cast<int>(inputImage[i]) - static_cast<int>(blurredImage[i]);
14.     // se adaugă diferența amplificată (amount * diff) la pixelul original, cu limitare la
[0, 255]
15.     int sharpened = static_cast<int>(inputImage[i]) + static_cast<int>(amount * diff);
16.     outputImage[i] = static_cast<unsigned char>(std::min(255, std::max(0, sharpened)));
17. }
```

În acest exemplu, imaginea originală este blurată folosind *BlurFilter* (pasul 1), apoi se calculează diferența și se aplică formula masei de neclaritate cu un factor de 1.5 (pasul 2). Rezultatul obținut în *outputImage* va conține imaginea ascuțită, având pixelii ajustați astfel încât *contururile și idetaliile să fie mai pronunțate.* Operația de clamping (std::min(255, std::max(0, sharpened))) asigură că valorile finale rămân în intervalul valid de intensitate 0 – 255.

Fig. 2.8.: Filtru de accentuare

În exemplul prezentat, imaginea din stânga reprezintă versiunea originală, în care detaliile și contururile sunt vizibile, dar pot părea ușor atenuate, mai ales în zonele de tranziție dintre obiecte și fundal. Aplicând filtrul Sharpen (dreapta), se observă că marginile cameramanului, ale trepiedului și ale elementelor din fundal devin mult mai bine definite, iar contrastul local este sporit semnificativ. Textura hainei și liniile subtile ale detaliilor devin mai pronunțate, iar impresia generală de claritate este vizibil îmbunătățită. Acest filtru reușește astfel să accentueze detaliile și să ofere imaginii un aspect mult mai bine conturat, fiind util atât în prelucrarea artistică, cât și în restaurarea sau îmbunătățirea calității imaginilor ușor neclare.

2.2.6. Efect mozaic (Mosaic)

Filtrul Mosaic (pixelare) reduce rezoluția **detaliilor imaginii prin împărțirea imaginii în blocuri dreptunghiulare și înlocuirea fiecărui bloc cu o culoare unică (cum ar fi media culorilor din acel bloc). Acest efect produce o imagine stilizată alcătuită din *„pixeli" mari* (blocuri uniforme de culoare), similar cu un mozaic sau cu pixelarea folosită pentru cenzurarea fețelor în materialele video.**

Pentru a obține efectul, algoritmul împarte imaginea în tile-uri (blocuri) de dimensiune fixă *tileSize × tileSize.* Pentru fiecare astfel de bloc, se calculează culoarea medie a tuturor pixelilor din interiorul lui, apoi toți pixelii acelui bloc sunt setați la acea culoare medie. În implementarea C++ s-au parcurs blocurile într-un dublu for (pe indexul de bloc vertical și orizontal), folosind OpenMP pentru paralelizare pe blocuri. Fragmentul următor prezintă calculul mediei pe un bloc și apoi atribuirea valorilor tuturor pixelilor din bloc la culoarea medie:

```
 1. // Calcul medie pe bloc (tileX, tileY)
 2. int avgR = 0, avgG = 0, avgB = 0, count = 0;
 3. for (int y = 0; y < tileSize; y++)
 4. {
 5.     int pixelY = tileY * tileSize + y;
 6.     if (pixelY >= height) break;
 7.     for (int x = 0; x < tileSize; ++x) {
 8.         int pixelX = tileX * tileSize + x;
 9.         if (pixelX >= width) break;
10.         int idx = (pixelY * width + pixelX) * 3;
11.         avgR += inputImage[idx];
12.         avgG += inputImage[idx+1];
13.         avgB += inputImage[idx+2];
14.         count++;
15.     }
16. }
17. avgR /= count; avgG /= count; avgB /= count;  // culoarea medie
18. // Aplicare culoare medie pe tot blocul
19. for (int y = 0; y < tileSize; y++)
20. {
21.     int pixelY = tileY * tileSize + y;
22.     if (pixelY >= height) break;
23.     for (int x = 0; x < tileSize; x++)
24.
25.         int pixelX = tileX * tileSize + x;
26.         if (pixelX >= width) break;
27.         int idx = (pixelY * width + pixelX) * 3;
28.         outputImage[idx]   = avgR;
29.         outputImage[idx+1] = avgG;
30.         outputImage[idx+2] = avgB;
31.     }
32. }
```

Fig. 2.9.: Efect mozaic

În exemplul din figura de mai sus s-a folosit un tile de 10 × 10 pixeli: fotografia originală (stânga) este tranformată în mozaic (dreapta), unde se remarcă apariția blocurilor pătrățoase uniforme. Cu cât dimensiunea tile-ului e mai mare, cu atât scade

rezoluția percepută a imaginii și efectul de pixelare este mai pronunțat (detaliile fine diispar). Filtrul mozaic are aplicabilitate practică la anonimizarea fețelor sau plăcuțelor auto din imagini, precum și ca efect vizual artistic retro.

2.2.7. Filtru Sepia

Filtrul Sepiia aplică imaginii un ton cald, maroniu – auriu, reminiscent de fotografiile vechi. Efectul sepia se poate obține printr-o transformare liniară a componentelor RGB ale imaginii color, urmată de o saturare a valorilor în intervalul valid. Formula standard folosește o matrice 3 × 3 de coeficienți predefiniiți pentru a calcula noile valori:

$$R' = 0.393R + 0.769G + 0.189B,$$
$$G' = 0.349R + 0.686G + 0.168B,$$
$$B' = 0.272R + 0.534G + 0.131B.$$

Această transformare crește contribuția canalului roșu și o scade pe cea a canalului albastru, generând tonalitatea specifică sepia (maro – roșiatic).

Implementarea C++ a filtrului a folosit exact formula de mai sus. S-a parcurs vectorul de pixelii și pentru fiecare pixel (R, G, B) s-au calculat în virgulă mobilă valorile noi, aplicând matricea sepia, apoi s-au convertit înapoi la octeți cu limitarea la 255. Fragmentul următor ilustrează calculul efectuat pe un pixel:

```
 1. static constexpr float sepiaMatrix[3][3] = {
 2.     {0.393f, 0.769f, 0.189f},
 3.     {0.349f, 0.686f, 0.168f},
 4.     {0.272f, 0.534f, 0.131f}
 5. };
 6. float newR = r * sepiaMatrix[0][0] + g * sepiaMatrix[0][1] + b * sepiaMatrix[0][2];
 7. float newG = r * sepiaMatrix[1][0] + g * sepiaMatrix[1][1] + b * sepiaMatrix[1][2];
 8. float newB = r * sepiaMatrix[2][0] + g * sepiaMatrix[2][1] + b * sepiaMatrix[2][2];
 9. // Clamp între 0 și 255
10. outputImage[i]   = static_cast<unsigned char>(std::clamp(newR, 0.0f, 255.0f));
11. outputImage[i+1] = static_cast<unsigned char>(std::clamp(newG, 0.0f, 255.0f));
12. outputImage[i+2] = static_cast<unsigned char>(std::clamp(newB, 0.0f, 255.0f));
```

Fig. 2.10.: Filtru sepia

Efectul vizual este vizibil în exemplul din figură: imaginea originală (stânga) capătă nuanțe sepia uniforme în versiune filtrată (dreapta). Culorile reci dispar aproape complet, fiind înlocuite de tonuri maronii. Acest filtru este adesea folosit pentru a crea un aspect *vintage* fotografiilor moderne.

2.2.8. Efect pictură în ulei (Oil Paint)

Filtrul Oil Paint (pictură în ulei) transformă fotografia într-o imagine cu aspect de tablou pictat, prin reducerea detaliilor și înlocuirea lor cu zone plate de culoare predominantă, ca și cum ar fi fost întinse cu pensula. Implementarea tipică a acestui filtru presupune analiza locală a imaginii: pentru fiecare pixel, se consideră o vecinătate (de exemplu un pătrat de r pixeli în fiecare direcție) și se determină care este culoarea predominantă în acea zonă, apoi pixelul este colorat cu acea culoare dominantă.

Culoarea „predominantă" se poate defini, de pildă, pe baza *intensității* (luminozității) pixelilor. În algoritmul proiectului, s-au folosit 256 de niveluri posibile de intensitate (0 = negru, 255 = alb). Pentru fiecare pixel central s-au inițializat trei histograme locale (una pentru fiecare canal R, G, B) indexate după intensitate. Apoi pentru fiecare pixel din vecinătatea defiinită (ex. o fereastră de 9 × 9 pixeli în jurul punctului curent, daca r = 4), s-a calculat intensitatea (media $\frac{R+G+B}{3}$) și s-a acumulat contribuția sa: histogramul de intensitate la acea valoare a fost incrementat, iar la acele indexuri de intensitate s-au însumat valorile R, G și B. Astfel, după ce se parcurg toți pixelii din vecinătate, vom ști pentru fiecare nivel de intensitate câți pixeli au avut acea luminozitate și suma valorilor de culoare asociate.

Pasul următor este identificarea intensității *dominante,* adică nivelul de gri care a apărut de cele mai multe ori în acea fereastră. S-a parcurs vectorul de frecvențe și s-a determinat indexul cu numărul maxim (dacă există mai multe, se poate alege primul găsit). Să notăm cu I_{dom} această intensitate dominantă. În final, pixelul central se colorează cu media culorilor tuturor pixelilor din vecinătate care au intensitatea I_{dom}. Aceste medii (sumă de R la intensitatea respectivă împărțită la numărul de pixeli la acel nivel, idem pentru G și B) vor da nuanța predominantă local. Dacă vecinătatea era omogenă, practic pixelul rămâne aproape neschimbat; dara era eterogenă devine reprezentativ pentru clasa majoritară de intensitate.

Implementarea în cod a urmat acești pași folosind structură paralelizată cu OpenMP (imaginea a fost împărțită pe linii între firele de execuție). Codul de mai jos

sintetizează procesul pentru un pixel (y,x) din imagine, accentuând pașii de numărare a frecvențelor și alegere a intensității dominante:

```
 1. // Inițializează contorizările pe intensități 0 - 255
 2. int localIntensityCount[256] = {0};
 3. int localSumR[256] = {0}, localSumG[256] = {0}, localSumB[256] = {0};
 4. // Parcurge vecinătatea (2 * filterRadius + 1) * (2 * filterRadius + 1)
 5. for (int offsetY = -filterRadius; offsetY <= filterRadius; offsetY++)
 6. {
 7.     int sampleY = std::clamp(y + offsetY, 0, height - 1);
 8.     for (int offsetX = -filterRadius; offsetX <= filterRadius; offsetX++)
 9.       {
10.         int sampleX = std::clamp(x + offsetX, 0, width-1);
11.         int sampleIdx = sampleY * width + sampleX;
12.         int intensity = intensityMap[sampleIdx];          // intensitatea pixelului vecin
13.         localIntensityCount[intensity]++;              // incrementează frecvența
14.         localSumR[intensity] += inputImage[sampleIdx * 3];  // acumulează valorile de
culoare
15.         localSumG[intensity] += inputImage[sampleIdx * 3 + 1];
16.         localSumB[intensity] += inputImage[sampleIdx * 3 + 2];
17.     }
18. }
19. // Determină intensitatea dominantă
20. int maxCount = 0, dominantIntensity = 0;
21. for (int i = 0; i < 256; i++)
22. {
23.     if (localIntensityCount[i] > maxCount)
24.       {
25.         maxCount = localIntensityCount[i];
26.         dominantIntensity = i;
27.     }
28. }
29. // Asignează pixelului central culoarea medie corespunzătoare intensității dominante
30. int outIndex = (y * width + x) * 3;
31. if (maxCount > 0)
32. {
33.     outputImage[outIndex]     = localSumR[dominantIntensity] / maxCount;
34.     outputImage[outIndex + 1] = localSumG[dominantIntensity] / maxCount;
35.     outputImage[outIndex + 2] = localSumB[dominantIntensity] / maxCount;
36. }
37. else
38. {
39.     // în caz extrem fără pixeli (fallback - nu se prea întâmplă)
40.     outputImage[outIndex]     = inputImage[outIndex];
41.     outputImage[outIndex + 1] = inputImage[outIndex + 1];
42.     outputImage[outIndex + 2] = inputImage[outIndex + 2];
43. }
```

Fig. 2.11.: Efect de pictură în ulei

În practică, filtrul produce suprafețe de culoare relativ uniforme, eliminate de zgomot, care dau impresia de tușe de pensule. În exemplul prezentat, imaginea originală

(stânga) păstrează detaliile și contururile clare ale Turnului Eiffel și ale peisajului înconjurător. După aplicarea filtrului (dreapta), detaliile fine dispar, iar suprafețele devin compuse din zone omogene de culoare, asemănătoare tușelor de pensulă. Nuanțele principale sunt păstrate, dar tranzițiile dintre culori sunt netezite, iar structura imaginii capătă o tentă picturală, ca într-un tablou realizat manual. Acest tip de filtru este folosit frecvent pentru a obține efecte vizuale deosebite, eliminând zgomotul și accentuând expresivitatea artistică a fotografiei.

Observație: Algoritmul descris este destul de intensiv computațional (pentru fiecare pixel vizitează toți pixelii din vecinătate); de aceea, folosirea OpenMP aici este crucială pentru a împărți liniile imaginii între nuclee și a reduce semnificativ timpul de execuție.

2.2.9. Efect glitch

Filtrul Glitch aplică imaginii un efect de *eroare digitală* deliberată, imitând artefactele vizuale care apar când un semnal video este corupt. Scopul este de a genera dungi, decalaje de linii, schimbări ale canalelor de culoare etc., care să confere imaginii un aspect haotic. Implementarea din proiect a inclus două tipuri de perturbări ale imaginii: (1) deplasarea aleatorie a unor coloane de pixeli și (2) permutarea aleatorie a canalelor de culoare la un subset de pixeli.

Mai întâi, imaginea de intrare este copiată ca atare în cea de ieșire (pentru a porni cu o versiune inițială identică). Apoi:

- Se alege un număr mic de coloane (de exemplu ~5% din total) care vor fi modificate. Pentru fiecare astfel de coloană aleasă aleator, se generează un shift (deplasare) întâmplătoare cu câțiva pixeli la stânga sau dreapta (ex: între -10 și +10 pixeli). Apoi, pentru fiecare rând al acelei coloane, pixelul din coloană este înlocuit cu pixelul de pe aceeași linie dar deplasat cu *shift* coloane mai în dreapta/stânga (cu tratament la margine). Acest lucru produce dungi verticale discontinue – fragmente din coloanele vecine sunt copiate pentru coloana curentă, ca și cum liniile ar fi fost ușor amestecate.
- Separat, se alege un număr mic de pixeli aleatori (de exemplu ~1% din pixeli) și pentru fiecare se selectează două canale de culoare la întâmplare (R, G sau B). Valorile acestor două canale sunt apoi *swap*-ate (interschimbate). De exemplu, pentru un pixel, se paote schimba valoarea roșie cu cea verde, sau

verde cu albastru etc. Acest lucru introduce puncte cu aberații cromatice – mici zone în care culoarea nu este corectă, ca și cum semnalul de culoare ar fi suferiit erori.

Fragmentul de cod ilustrează ambele componente ale efectului glitch: deplasarea de coloane (bucată din prima buclă paralelizată) și schimbul de canale (a doua buclă):

```
 1. // Deplasare aleatorie a unor coloane
 2. std::mt19937 gen(seed + omp_get_thread_num());
 3. std::uniform_int_distribution<> colDist(0, width - 1);
 4. std::uniform_int_distribution<> shiftDist(-shiftAmount, shiftAmount);
 5. #pragma omp for
 6. for (int i = 0; i < numColumnsToShift; i++)
 7. {
 8.     int column = colDist(gen);
 9.     int shift = shiftDist(gen);
10.     for (int y = 0; y < height; y++)
11.       {
12.         int srcX = std::clamp(column + shift, 0, width - 1);
13.         int dstIndex = (y * width + column) * channels;
14.         int srcIndex = (y * width + srcX) * channels;
15.         // Copiază pixelul de pe coloana deplasată
16.         std::copy(outputImage + srcIndex, outputImage + srcIndex + channels, outputImage +
dstIndex);
17.     }
18. }
19. // Schimb aleatoriu al canalelor la un număr de pixeli
20. std::uniform_int_distribution<> pixDist(0, width * height - 1);
21. std::uniform_int_distribution<> chDist(0, channels - 1);
22. #pragma omp for
23. for (int i = 0; i < numPixelsToAlter; i++)
24. {
25.     int pixelIndex = pixDist(gen) * channels;
26.     int c1 = chDist(gen);
27.     int c2 = chDist(gen);
28.     std::swap(outputImage[pixelIndex + c1], outputImage[pixelIndex + c2]);
29. }
```

Fig. 2.11.: Efect glitch

Efectul filtrului „Glitch" poate fi observat clar în exemplul de mai sus. Imaginea originală (stânga) prezintă contururi bine definite și o structură normală a detaliilor. După aplicarea filtrului Glitch (dreapta), apar deformări vizibile, cum ar fi linii verticale striate, ușor deplasate și „fracturate", precum și mici erori vizuale pe anumite porțiuni ale imaginii. Aceste artefacte creează impresia de eroare digitală, ca și cum datele imaginii ar fi fost corupte sau transmise defectuos. Intensitatea și tipul efectelor glitch

pot fi ajustate prin parametri specifici ai filtrului (numărul și poziția coloanelor deplasate, nivelul de distorsiune etc.). Scopul acestui filtru este predominant artistic, pentru a adăuga un aspect neobișnuit și experimental imaginilor procesate.

2.2.10. Efect caleidoscop (Kaleidoscope)

Filtrul Kaleidoscope creează un efect de simetrie multiplă în imagine, ca și cum am privi imaginea printr-un caleidoscop optic. Principiul este că imaginea este *repetată și oglindită* de mai multe ori în jurul unui punct central, generând un model simetric circular. Implementarea noastră a folosit o abordare geometrică în coordonate polare: pentru fiecare pixel al imaginii de ieșire, s-a determinat de unde să preia culoarea din imaginea originală astfel încât să rezulte acest efect simetric.

Concret, algoritmul consideră centrul imaginii ca origine și calculează pentru fiecare pixel de coordonate (x, y) vectorul față de centru (dx, dy), exprimându-l în coordonate polare – adica unghiul $\theta = \arctan 2(dy, dx)$ și raza $r = \sqrt{dx^2 + dy^2}$. Apoi, unghiul θ este transformat prin *oglindire modulară:* se împarte cercul 360° în N segmente egale (unde N este numărul de reflexii dorite, de exemplu 6 sau 8). În implementare, s-a folosit *numReflections = 6* pentru a obține un model hexagonal simetric. Se calculează unghiul segment $\alpha = \frac{360^\circ}{N}$ (în radiaini, $\frac{\pi}{N}$) și apoi noul unghi *mirrored* se obține practic ca $\theta' = \theta \; mod \; \alpha$, aducând unghiul în primul segment, iar daca θ era negativ, se ia simetricul pozitiv. Cu alte cuvinte, indiferent ce unghi avea pixelul inițial, el va fi „pliat" în intervalul $[0, \alpha)$.

După obținerea unghiului oglindit θ', se convertește înapoi la coordonate carteziene:

$$x' = centerX + r\cos\theta',$$
$$y' = centerY + r\sin\theta',$$

care reprezintă coordonata din imaginea originală de la care vom prelua culoarea. Dacă aceste coordonate cad în afara limitelor (din cauza rotunjirilor), se vor satura la margini. În final, pixelul de ieșire (x, y) primește valoarea pixelului original (x', y').

Codul sursă a realizat aceste operații în interiorul a două bucle for pe y și x, folosind funcții matematice din *<cmath>* și efectuand *memcpy* pentru a copia tripletul de culori de la sursă la destinație. Mai jos este reprodusă partea principală a algoritmului caleidoscop:

```
1. float dx = x - centerX;
2. float dy = y - centerY;
3. float angle = atan2f(dy, dx);
```

```
 4. float radius = sqrtf(dx * dx + dy * dy);
 5.
 6. // oglindire unghi în primul segment [0, 2π/numReflections)
 7. float segmentAngle = M_PI / numReflections;
 8. float mirroredAngle = fmodf(fabsf(angle), segmentAngle);
 9.
10. if (angle < 0) mirroredAngle = -mirroredAngle;
11.
12. // coordonate oglindite
13. int mirroredX = static_cast<int>(centerX + radius * cosf(mirroredAngle));
14. int mirroredY = static_cast<int>(centerY + radius * sinf(mirroredAngle));
15.
16. // clamp la limitele imaginii sursă
17. mirroredX = std::clamp(mirroredX, 0, width - 1);
18. mirroredY = std::clamp(mirroredY, 0, height - 1);
19.
20. // copiere pixel de la (mirroredX, mirroredY) la (x, y)
21. int srcIndex = (mirroredY * width + mirroredX) * channels;
22. int dstIndex = (y * width + x) * channels;
23.
24. memcpy(&outputImage[dstIndex], &inputImage[srcIndex], channels);
```

Fig. 2.12.: Efect caleidoscop

Efectul creat de filtrul „Kaleidoscope" este ușor de remarcat în figura de mai sus. Imaginea originală (stânga) surprinde o floare în detaliu, cu culori și forme naturale. După aplicarea filtrului (dreapta), compoziția vizuală se transformă spectaculos: petalele și centrul florii sunt multiplicate radial, creând un model geoemtric simetric, asemănător unui caleidoscop optic. Textura și culorile inițiale sunt păstrate, dar structura imaginii devine una abstractă, cu repetiții fractale și segmente care convers spre mijloc. Acest filtru are rol exclusiv artistic, evidențiind posibilitățile creative ale prelucrării geometrice a pixelilor pentru generarea unor compoziții vizuale unice și atractive.

2.2.11. Filtrul Sobel

Filtrul Sobel este un operator de detecție a marginilor bazat pe derivata imaginii. El calculează o aproximație a gradientului intensității pentru fiecare pixel din imagine, evidențiind zonele cu variații bruște de intensitate – adică potențiale margini. Operatorul Sobel folosește două măști de convoluție 3 × 3, una pentru direcția orizontală și alta pentru direcția verticală, pentru a estima derivata parțială pe fiecare axă. Rezultatul combină aceste derivate (gradientul pe axa x și pe axa y) astfel încât

valoarea mare a gradientului indică prezența unei margini în acea poziție a imaginii. Sobel este utilizat pe scară largă în procesarea imaginilor datorită simplității și vitezei sale, fiind adecvat pentru detectarea contururilor de bază în timp real. Totuși, deoarece nu iniclude un mecanism complex de filtrare a zgomotului sau de subțiere a marginilor, rezultatele pot avea muchii mai groase și potențial mai zgomotoase comparativ cu metode mai avansate.

Măștile de convoluție Sobel sunt definite astfel: pentru direcția orizontală (gradient Gx) se folosește nucleul:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix},$$

Iar pentru direcția verticală (gradient Gy) nucleul este:

$$S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Convoluția imaginii I cu aceste nuclee produce două imagini gradient, $G_x = I \cdot S_x$ și $G_y = I \cdot S_y$, care reprezintă aproximări ale derivatelor parțiale $\frac{\partial I}{\partial x}$ respectiv $\frac{\partial I}{\partial y}$. Pe baza lor se calculează *magnitudinea gradientului* în fiecare pixel: $G = \sqrt{G_x^2 + G_y^2}$.

Rezultatul filtrului Sobel este în general imaginea magnitudinii gradientului (eventual scalată la intervalul [0, 255] pentru afișare). Opțional, se poate calcula și *direcția gradientului* în fiecare punct, $\theta = \arctan\frac{G_y}{G_x}$, care oferă orientarea locală a conturului detectat.

Algoritmul Sobel. În esență, aplicarea filtrului Sobel presupune parcurgerea întregii imagini și aplicarea celor două filtre de convoluție. Pașii principali ai algoritmului sunt următorii:

1. *Pregătire:* Dacă imaginea este color, se convertesc datele în gri (luminanță) pentru a lucra cu un singur canal de intensitate.
2. *Convoluții:* Se aplică convoluția imaginii cu kernelul S_x (detecția marginilor verticale) și cu kernelul S_y (detecția marginilor orizontale), obținându-se gradientul pe direcția x și y pentru fiecare pixel.
3. *Calculul magnitudinii:* Pentru fiecare poziție (x, y), pe baza celor două componente $G_x(x, y)$ și $G_y(x, y)$, se calculează magnitudinea gradientului $G(x, y) = \sqrt{G_x^2 + G_y^2}$. Aceast[valoare indică „forța" marginii în acel punct.
4. *Post-procesare:* Valorile rezultate pot fi normalizate sau tăiate la 255 (saturație) dacă depășesc intervalul de afișare. De asemenea, se poate aplica

opțional un prag *(threshold)* pentru a obține un rezultat binar (marginile puternice sunt evidențiate, zgomotul fiind eliminat). În mod tradițional, filtrul Sobel simplu nu include o etapă de binarizare automată, lăsând la latitudinea utilizatorului alegerea unui prag adecvat.

Mai jos este prezentat un fragment de cod C++ din implementarea metodei Sobel, care ilustrează pașii de mai sus. Codul definește nucleele orizontal și vertical, realizează convoluția imaginii (ținând cont de muchiile imaginii prin replicare de pixeli de la margine) și calculează magnitudinea gradientului, scriind rezultatul în pixelii imaginii de ieșire:

```
 1.      // Kernel-urile filtrului Sobel
 2.      static constexpr int kernelX[3][3] = { { -1, 0, 1 },{ -2, 0, 2 },{ -1, 0, 1 } };
 3.      static constexpr int kernelY[3][3] = { { -1, -2, -1 },{ 0, 0, 0 },{ 1, 2, 1 } };
 4.
 5.      const int size = width * height * channels;
 6.
 7. #pragma omp parallel for
 8.      for (int y = 0; y < height; y++)
 9.      {
10.              // Pre-alocam vectorii
11.              std::vector<float> sumX(channels, 0.0f);
12.              std::vector<float> sumY(channels, 0.0f);
13.              for (int x = 0; x < width; x++)
14.              {
15.                      // Se resetează vectorii pentru fiecare pixel
16.                      std::fill(sumX.begin(), sumX.end(), 0.0f);
17.                      std::fill(sumY.begin(), sumY.end(), 0.0f);
18.
19.                      // Convoluție cu verificarea limitelor
20.                      for (int i = -1; i <= 1; i++)
21.                      {
22.                              int sampleY = y + i;
23.                              if (sampleY < 0) sampleY = 0;
24.                              else if (sampleY >= height) sampleY = height - 1;
25.
26.                              for (int j = -1; j <= 1; j++)
27.                              {
28.                                      int sampleX = x + j;
29.                                      if (sampleX < 0) sampleX = 0;
30.                                      else if (sampleX >= width) sampleX = width - 1;
31.
32.                                      int sampleIndex = (sampleY * width + sampleX) *
channels;
33.
34.                                      for (int c = 0; c < channels; c++)
35.                                      {
36.                                              float pixel = inputImage[sampleIndex
+ c];
37.                                              sumX[c] += pixel * kernelX[i + 1][j +
1];
38.                                              sumY[c] += pixel * kernelY[i + 1][j +
1];
39.                                      }
40.                              }
41.                      }
42.
43.                      // Calculează magnitudinea gradientului și scrie în imaginea
destinație
44.                      int index = (y * width + x) * channels;
45.                      for (int c = 0; c < channels; c++)
```

```
46.                         {
47.                                 float magnitude = std::sqrt(sumX[c] * sumX[c] + sumY[c] *
sumY[c]);
48.                                 if (magnitude > 255.0f)
49.                                 {
50.                                         magnitude = 255.0f;
51.                                 }
52.                                 outputImage[index + c] = static_cast<unsigned
char>(magnitude);
53.                         }
54.                 }
55.         }
```

Acest cod parcurge fiecare pixel al imaginii de intrare și aplică cei doi kerneli Sobel (orizontal și vertical) folosind bucle imbricate. Pentru fiecare poziție (x, y), se acumulează separat sumele ponderate *sumX* și *sumY* corespunzătoare convoluției cu kernelul orizontal, respectiv vertical. În interiorul buclelor, există verificări de *limite* (liniile *if (sampleY < 0) ...* etc.) care asigură că indecșii nu ies în afara marginii – la margini, pixelii de dincolo de limită sunt înlocuiți cu ultimul pixel valid (practic se duplică marginea imaginii). După convoluție, codul calculează *magnitudinea gradientului (sqrt(sumX * sumX + sumY * sumY))* pentru fiecare canal de culoare al pixelului. Dacă valoarea depășește 255 (limita maximă pentru un pixel de 8 biți), este plafonată la 255. În final, valorile obținute sunt convertite la *unsigned char* și inscrise în buffer-ul imaginii de ieșire. Prin aceste operații, imaginea rezultată va conține intensități mari (pixel luminos) acolo unde exista o schimbare semnificativă în imaginea originală – adică la margini și contururi.

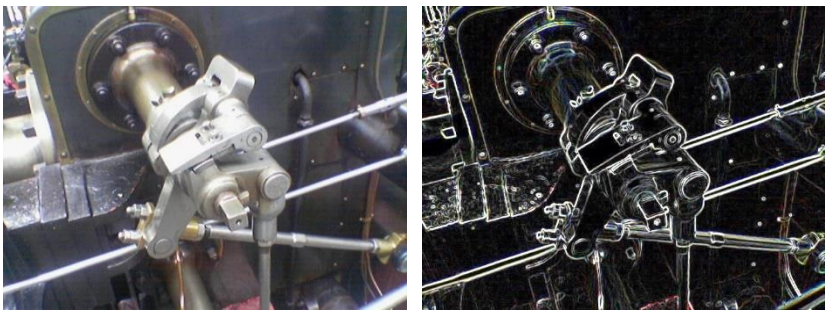

Fig. 2.13.: Filtru Sobel

În figura de mai sus, este ilustrat rezultatul filtrului Sobel aplicat asupra unei imagini reale. Se poate observa că operatorul Sobel a detectat contururile principalelor componente (muchiile pieselor metalice și ale îmbinărilor). Marginile apar evidențiate prin linii albicioase pe fundal închis, având o grosime de câțiva pixeli. Deși contururile obiectelor sunt în mare parte vizibile, ele apar ușor estompate și discontinue din cauza

absenței unui prag adaptiv sau a unei etape de subțiere – acesta este un efect tipic pentru filtrul Sobel, care produce adesea margini mai groase și cu zgomot dacă imaginea are texturi fine sau variații rapide de intensitate.

2.2.12. Filtrul Canny

Filtrul Canny reprezintă un algoritm de detecție a marginilor mult mai avansat, ce include o secvență de etape pentru a identifica *toate marginile relevante cu precizie sporită*, minimizând în același timp detectările false. Dezvoltat de John F. Canny în 1986, acest algoritm urmărește satisfacerea a trei criterii de performanță pentru detecția optimă a muchiilor: (1) *rată de detecție ridicată* – detectarea cât mai completă a marginilor reale (puține margini ratate și puține alarme false), (2) *localizare precisă* – poziția marginilor detectate să fie cât mai aproape de poziția reală a contururilor din imagine, și (3) *unică detectare* – un singur răspuns (o singură linie) per margine reală. Pentru a îndeplini aceste condiții, algoritmul Canny realizează o serie de prelucrări complexe, combinând filtrarea pentru reducerea zgomotului cu tehnici adaptative de evidențiere și conectare a marginilor. În ciuda complexității sale, filtrul Canny a devenit unul dintre cei mai populari detectori de muchii, datorită acurateței ridicate a rezultatelor și a teoriei bine fundamentate din spatele său.

La baza algoritmului Canny se află calcule smilare cu cele din Sobel (derivate parțiale și magnitudinea gradientului), combinate însă cu *filtrare gaussiană* și praguri adaptative. Prima etapă implica netezirea imaginii cu un filtru Gaussian (de obicei de mărime 5 × 5, $\sigma \approx 1$) pentru a atenua zgomotul din date. Formula nucleului Gaussian este: $G(x,y) = \frac{1}{2\pi\sigma^2}\exp(-\frac{x^2+y^2}{2\sigma^2})$, iar convoluția imaginii cu acest nucleu realizează o *filtrare low-pass* (înlăturarea variațiilor bruște considerate zgomot). Ulterior, se calculează imaginile gradient G_x și G_y (de exemplu, folosind tot operatori de tip Sobel sau derivați ai unui Gaussian), apoi magnitudinea gradientului $G = \sqrt{G_x^2 + G_y^2}$ și unghiul (direcția) gradientului $\theta = \arctan\frac{G_y}{G_x}$ pentru fiecare pixel. În loc să marcheze imediat toți pixelii cu gradient mare drept margini, Canny introduce două praguri, *pragul jos* și *pragul înalt*, notate de obicei T_{low} și T_{high}, cu $T_{low} < T_{high}$. Valorile acestor praguri pot fi alese absolut (de exemplu $T_{low} = 50$, $T_{high} = 150$ pe o scară 0 – 255) sau relativ, adesea proporțional cu maximul din imagine (în implementarea dată, $T_{low} = 0.1 \cdot G_{max}$ și $T_{high} = 0.3 \cdot G_{max}$, unde G_{max} este cel mai mare gradient din imagine). Aceste praguri permit clasificarea pixelilor cu gradient mare drept „margini puternice" și a celor cu

gradient intermediar drept „margini posibile" (cele cu gradient sub pragul mic sunt considerate zgomot și eliminate).

Spre deosebire de Sobel, detectorul Canny urmează o serie de 5 pași secvențiali pentru a obține un set final de margini precise și conectate:

1. *Filtrare gaussiana (reduce zgomotul):* se aplică un filtru Gaussian imaginii inițiale, rezultând o versiune netezită a acesteia. Acest pas elimină zgomotul de înaltă frecvență care ar putea produce detecții de margini false.
2. *Calculul gradientului:* se calculează derivatele parțiale G_x și G_y (de obicei cu operator Sobel pe imaginea netezită) și din acestea magnitudinea gradientului G și direcția θ pentru fiecare pixel. Punctele cu gradient ridicat (valori mari în G) sunt candidate pentru a fi margini.
3. *Suprimarea non-maximelor (NMS – non-maximum suppresion):* fiecare pixel candidat este comparat cu vecinii săi de-a lungul direcției gradientului θ. Dacă valoarea sa $G(x, y)$ nu este maxima locală pe direcția respectivă, ea este suprimată (setată la zero). Această etapă subțiază marginile – ideal, rezultând contururi de lățime 1 pixel – și elimină răspunsurile multiple în apropierea aceleiași muchii.
4. *Pragul dublu (double thresholding):* se aplică cele două praguri T_{high} și T_{low} asupra imaginii gradient filtrate de la pasul anterior. Pixelii cu $G(x, y) \geq T_{high}$ sunt marcați ca *margini puternice* (vor apărea sigur în rezultatul final), cei cu $G(x, y) < T_{low}$ sunt considerați non-margini (sunt eliminați), iar pixelii cu $T_{low} \leq G(x, y)$ sunt marcați ca *margini slabe* – potențiale margini care necesită confirmare în pasul următor.
5. *Legarea mariginilor prin histerezis:* ultimul pas conectează marginile slabe de cele puternice, asigurând continuitatea contururilor. Orice pixel marcat ca margine slabă care este adiacent (8 – conectivitate) cu cel puțin un pixel de margine puternică va fi păstrat și promovat la margine (devine parte din conturul final): În schimb, pixelii slabi izolați (fără vecini puternici) sunt considerați zgomot și sunt înlăturați. Acest procedeu, numiit *histerezis*, urmărește să păstreze numai marginile semnificative șii conectate și să elimine detectările de margini sporadice sau datorate zgomotului.

Mai jos este redat un fragment de cod C++ care implementează pașii algoritmului Canny descriși anterior. Se poate observa structurarea clară în etape nuemrotate și utilizarea pragurilor dublu adaptativie și a urmăririi marginilor:

```
 1.       // 1. Convertește în tonuri de gri
 2.       std::vector<unsigned char> grayImage(width * height);
 3.
 4. #pragma omp parallel for
 5.       for (int i = 0; i < width * height; i++)
 6.       {
 7.                 if (channels > 1)
 8.                 {
 9.                           const unsigned char* pixel = inputImage + (i * channels);
10.                           grayImage[i] = static_cast<unsigned char>(0.299 * pixel[0] + 0.587
* pixel[1] + 0.114 * pixel[2]);
11.                 }
12.                 else
13.                 {
14.                           std::copy_n(inputImage + i, 1, grayImage.begin() + i);
15.                 }
16.       }
17.
18.       // 2. Aplică Blur Gaussian pentru reducerea zgomotului
19.       std::vector<unsigned char> blurredImage(width * height);
20.       ApplyGaussianBlur(grayImage.data(), blurredImage.data(), width, height);
21.
22.       // 3. Calculează magnitudinea și direcția gradientului
23.       std::vector<float> gradientMagnitude(width * height);
24.       std::vector<float> gradientDirection(width * height);
25.       CalculateGradients(blurredImage.data(), gradientMagnitude.data(),
gradientDirection.data(), width, height);
26.
27.       // 4. Non-Maximum Suppression
28.       std::vector<unsigned char> suppressedImage(width * height, 0);
29.       NonMaximumSuppression(gradientMagnitude.data(), gradientDirection.data(),
suppressedImage.data(), width, height);
30.
31.       // 5. Aplicarea pragului dublu și conectarea muchiilor folosind histerezis
32.       std::vector<unsigned char> finalEdges(width * height, 0);
33.       DoubleThresholdAndHysteresis(suppressedImage.data(), finalEdges.data(), width,
height);
34.
35.       // 6. Copiază în imaginea destinație, expandând la numarul original de canale
36. #pragma omp parallel for
37.       for (int i = 0; i < width * height; i++)
38.       {
39.                 for (int c = 0; c < channels; c++)
40.                 {
41.                           outputImage[i * channels + c] = finalEdges[i];
42.                 }
43.       }
```

Acest fragment evidențiază implementarea practică a algoritmului Canny. În prima parte se realizează conversia în tonuri de gri. Urmează filtrarea Gaussiana (pasul 2), prin apelul funcției *ApplyGaussianBlur* – aceasta aplică un kernel Gaussian 5 × 5, așa cum este definit în codul sursă al funcției. După obținerea imaginii netezite, se calculează matricile *gradientMagnitude* și *gradientDirection* (pasul 3) prin derivare – în implementare, funcția *CalculateGradients* folosește la rândul ei un operator Sobel intern pentru a obține G_x, G_y și apoi calculează magnitudinea și direcția gradientului pentru

fiecare pixel. Pasul 4, Non-Maximum Suppresion, este implementat de funcția *NonMaximumSuppression*, care parcurge fiecare pixel (exceptând marginile imaginii) și compară valoarea gradientului său cu a pixelilor vecini de-a lungul direcției gradientului. În cod, direcția (un unghi în grade) este normalizată la intervalul [0, 180) și împărțită în patru intervale pentru a identifica corect veciinii „din stânga" și „din dreapta" de pe direcția respectivă. Dacă magnitudinea curentă nu este mai mare decât a vecinilor săi aliniați pe direcția muchiei, pixelul este suprimat (valoarea sa este setată la 0 în *outputImage*). Pasul 5 realizează pragul dublu și histerezisul. Funcția *DoubleThresholdAndHysteresis* stabilește mai întâi valorile pragurilor T_low și T_high ca fracțiuni din gradientul maxim. Apoi, copiază în *outputImage* valorile din *inputImage* (corespunzătoare imaginii obținute dupa NMS, deci conține magnitudini filtrate). Pixelii cu valoare peste T_{high} sunt marcați direct ca margini puternice. Pixelii cu valoarea sub T_{low} sunt considerați non-marginali și eliminați. Pentru pixelii cu valoarea intermediară ($[T_{low}, T_{high})$), codul verifică vecinătatea de 8 pixeli a fiecăruia: dacă cel puțin un vecin este deja marcat ca margine puternică, atunci pixelul curent este păstrat, altfel este eliminat. Această procedură leagă segmentele de margini între ele, asigurând continuitatea contururilor în rezultatul final. La finalul pasului 5, matricea *finalEdges* conține valoarea 255 pentru pixeli ce aparțin unor margini confirmate și 0 pentru fundal.

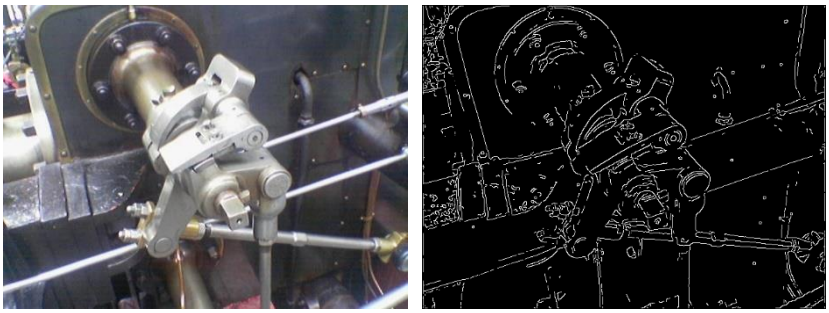

Fig. 2.14.: Filtru Canny

Figura de mai sus prezintă rezultatul aplicării detectorului Canny asupra aceleiași imagini de intrare folosite anterior. Se remarcă faptul că algoritmul Canny produce margini mult mai subțiri și mai curate: contururile sunt trasate prin linii albe de un singur pixel grosime, continue de-a lungul componentelor obiectului. Prin filtrarea Gaussiană inițială, multe dintre detecțiile false de la Sobel (de ex. texturi fine sau zgomot) au fost eliminate, iar prin pragul dublu și histerezis au fost păstrate doar marginile semnificative, conectate între ele. În consecință, imaginea rezultată conține un set mai redus de pixeli activi, dar care conturează mai fidel și precis obiectele din scenă.

2.2.13. Compararea filtrelor Sobel și Canny

Deși ambele tehnici urmăresc evidențierea contururilor într-o imagine, filtrul Sobel și detectorul Canny diferă semnificativ ca performanță și complexitate. Precizia marginilor detectate de algoritmul Canny este, în general, mai mare decât în cazul Sobel. Canny utilizează etapele de suprimare a non-maximelor și pragurile histerezis pentru a obține contururi subțiri (un singur pixel) și bine localizate, foarte apropiate de poziția reală a muchiilor din imagine. Prin contrast, Sobel tinde să producă margini mai groase și mai difuze, deoarece nu are un mecanism de subțiere – practic, răspunsul în gradient al filtrului Sobel se întinde pe mai mulți pixeli în vecinătatea unei margini. Un avantaj al Sobel este că furnizează direct informația de orientare a marginii (unghiul gradientului) pentru fiecare pixel, însă Canny oferă și el această informație implicit în calculul direcției gradientului, combinată cu o mai bună delimitare spațială a conturului. În termeni calitativi, marginile detectate de Canny sunt mai fine și urmează mai exact formele obiectelor, pe când cele de Sobel pot apărea fragmentate sau ușor decalate față de muchia reală.

Referitor la sensibilitatea la zgomot, filtrul Sobel este mai vulnerabil la variațiile de intensitate nesemnificative. Fără o filtrare prealabilă, Sobel poate evidenția și mici fluctuații care nu reprezintă margini reale, rezultând într-un număr mai mare de detecții false pe imagini zgomotoase. Canny abordează explicit problema zgomotului prin netezirea Gaussiana inițială și prin pragul inferior T_{low} care filtrează răspunsurile slabe. Astfel, Canny prezintă o robustețe sporită la zgomot – multe dintre falsele contururi cauzate de textură fină sau artefacte de imagine sunt eliminate în etapele algoritmului. Conform literaturii, operatorul Sobel oferă doar o reducere limitată a zgomotului prin natura sa (măștile 3 × 3 având un ușor efect de netezire), pe când Canny integrează explicit un filtru Gaussian și praguri adaptative pentru a elimina zgomotul.

Din punct de vedere al complexității algoritmice, Sobel este mul mai simplu. El realizează doar câteva operații de convoluție (două filtre 3 × 3 aplicate pe întreaga imagine) și calcule aritmetice pixel-cu-pixel, având o complexitate aproximativ O(N) (proporțională cu numărul de pixeli N). Implementarea Sobel este directă și eficientă, fiind adecvată pentru aplicații în timp real sau dispozitive cu putere de calcul limitată. În schimb, algoritmul Canny implică multiple etape secvențiale – filtrare Gaussiană (convoluție pe fereastră mai mare, ex. 5 × 5), calculul gradientului (ex. operator Sobel

intern), apoi parcurgeri suplimentare ale imaginii pentru NMS și pentru aplicația pragurilor și a histerezisului. Aceste etape suplimentare cresc timpul de execuție și complexitatea computațională a lui Canny (de ordinul O(4N) sau mai mare, în funcție de implementare). În practică, Canny este mai lent decât Sobel și consumă mai multe resurse, ceea ce poate fi un dezavantaj în sisteme unde viteza este critică. Totuși, pentru multe aplicații, costul computațional suplimentar este justificat de acuratețea superioară a detecției.

În ceea ce privește domeniile de aplicabilitate, alegerea între Sobel și Canny depinde de cerințele specifice ale problemei. Operatorul Sobel este adecvat pentru detecții rapide sau sarcini preliminare, unde se dorește evidențierea aproximativă a contururilor principale. De exemplu, Sobel poate fi utilizat în sisteme ebeded simple, pentru detecția unor muchii evidente sau ca etapă în extragerea unor trăsături inițiale, unde un timp de calcul foarte mic este esențial. Pe de altă parte, detectorul Canny este preferat în aplicații ce necesită precizie ridicată și o bună separare a obiectelor de fond – cum ar fi analiza de imagini medicale, recunoașterea formelor și a obiectelor, sistemele de viziune artificială din robotică sau vehicule autonome, unde contururile exacte și continue sunt critice. Canny permite ajustarea parametrilor (sigma filtrului Gaussian, pragurile) în funcție de caracteristicile datelor și cerințele aplicației, oferind astfel flexibilitate mai mare decât Sobel. În rezumat, Sobel rămâne o unealtă utilă pentru detecții de bază și când resursele sunt limitate, însă pentru rezultate de înaltă calitate în detecția marginilor, algoritmul Canny este, de regulă, opțiunea recomandată, oferind contururi mai curate și o performanță robustă la zgomot și variații de intensitate.

Capitolul 3

3. Tehnologii folosite

3.1. C++ și OpenMP

Limbajul C++ a fost ales pentru implementarea nucleului de procesare a imaginilor datorită eficienței sale ridicate și controlului direct asupra memoriei. C++ este recunoscut ca fiind foarte rapid comparativ cu limbaje de nivel înalt, permițând execuția operațiilor intensive într-un timp redus. Totodată, suportul pentru programarea orientată pe obiecte (OOP) în C++ face dezvoltarea mai ușoară și structurată, facilitând organizarea codului în clase și module reutilizabile. În contextul proiectului, aceste caracteristici permit implementarea unor algoritmi complecși de prelucrare a imaginilor (precum filtre grafice) într0un mod eficient, menținând în același timp un cod clar și modular.

Pentru a valorifica la maximum puterea procesoarelor multi-core, s-a utilizat OpenMP ca soluție de paralelizare a calculului în C++. OpenMP este un API larg utilizat pentru programarea paralelă în limbajele C/C++, permițând dezvoltatorilor să scrie cod paralel în mod simplu și eficient prin adăugarea unor directive de compilator în codul existent. Practic, secțiunile de cod intens computaționale – cum ar fi parcurgerea pixelilor unei imagini pentru aplicarea unui filtru – au fost marcate cu directive OpenMP (*#pragma omp parallel for* etc.), astfel încât să fie distribuite pe mai multe thread-uri. Această abordare a condus la reducerea timpului de procesare a imaginilor, apropiindu-se de o scalare aproape liniară cu numărul de nuclee de procesor disponibile. În implementare, s-au folosit funcții din bibliotecă (*omp_get_wtime()*) pentru a măsura durata execuției paralelizate, confirmând câștigul de performanță obținut prin multithreading.

Codul C++ rezultat a fost ocmpilat sub forma unei biblioteci dinamice (Dynamic-Link Library) pentru a putea fi integrat ușor cu celelalte componente ale sistemului. S-a folosit Cmake ca instrument de automatizare a build-ului, ceea ce a permis o construcție multiplatformă a bibliotecii native. Cmake generează fișierele de proiect necesare atât pentru compilatorul MSVC pe Windows, producând un fișier *.dll* cât și pentru compilatoarele pe Linux (gcc/clang), producând un fișier *.so* corespunzător. Acest lucru

asigură portabilitatea soluției și posibilitatea rulării serviciului de procesare a imaginilor pe diferite sisteme de operare. De asemenea, codul C++ a fost scris având în vedere managementul manual al memoriei: memoria alocată pentru imaginea procesată este eliberată prin funcții exportate (*FreeMemory)* ce pot fi invocate din exterior, prevenind scurgerile de memorie.

Pentru a garanta calitatea și corectitudinea modulului nativ, s-au implementat teste dedicate. Testarea unitară a fost folosită pentru a verifica funcțiile de bază. S-au realizat și teste de performanță, dar și teste de integrare care au verificat interacțiunea dintre biblioteca C++ și aplicația .NET (prin încărcarea DLL-ului și apelarea funcțiilor sale din codul C#), confirmând că schimbul de date (imaginea procesată și valorile de întoarcere) se realizează fără erori.

3.2. .NET 8 (C# și REST API)

Pentru dezvoltarea părții de backend s-a utilizat platforma .NET 8, împreună cu limbajul C#. .NET 8 este o platforma modernă, multi-platformă, care permite rularea aplicațiilor pe Windows și Linux, oferind totodată performanțe îmbunătățite față de versiunile anterioare și suport pe termen lung (LTS). Aplicația backend este construită ca un serviciu web de tip REST API, folosind framework-ul ASP.NET Core. Arhitectura Rest (Representational State Transfer) definește un set de principii pentru dezvoltarea de servicii web, bazate pe comunicarea stateless între client și server, prin intermediul protocoalelor standard HTTP. În practică, API-ul expune rute (endpoints) prin care clientul (aplicația Angular) poate trimite cereri pentru încărcarea și procesarea imaginilor sau obținerea rezultatelor. Fiecare cerere HTPP (POST pentru încărcare/procesare, GET pentru obținerea rezultatelor, etc.) produce un răspuns JSON, asigurând o interfață clară și ușor de utilizat de către diverși clienți.

Un aspect important al backend-ului .NET este integrarea cu biblioteca nativă C++ prin mecanismul Platform Invocation Services (P/Invoke). P/Invoke este o facilitate oferită de runtime-ul .NET care permite codului administrat (C#) să apeleze cod nativ (C/C++) dintr-o bibliotecă dyanmic-link. În cadrul proiectului s-au definit prototipurile funcțiilor native în codul C# folosind noul atribut *[LibraryImport]* (disponibil în .NET 7+ ca înlocuitor al vechiului *DllImport)*. De exemplu, clasa statică *ImageProcessor* (din C#) declară metoda *ApplyFilter* ca funcție externă, legată la biblioteca nativă *ImagesProcessor.dll* (sau *libImagesProcessor.so* pe Linux). Astfel, apelând metoda C#

ImageProcessor.GetFilteredImageData(...), se realizează în mod transparent un apel al funcției C++ corespunzătoare, datele binare ale imaginii fiind trimise către codul nativ, care procesează imaginea și întoarce rezultatul tot sub formă de buffer. Conversia și marshaling-ul parametrilor se efectuează automat: tabloul de octeți (byte[]) din C# este pasat ca pointer către funcția C++, iar la întoarcere se folosește *Marshal.Copy* pentru a copia datele procesate din memorie nativă în memorie administrată C#. Pentru a evita scurgerile de memorie, metoda C# invocă ulterior funcția nativă *FreeMemory*, care eliberează buffer-ul alocat în C++ (după ce datele au fost transferate înapoi în mediul managed).

De asemenea, backend-ul .NET 8 utilizează SignalR pentru a oferi actualizări în timp real către clientul web. SignalR este o bibliotecă open-source pentru ASP.NET care facilitează comunicarea bi-direcțională real-time între server și client. În proiect, SignalR este folosit pentru a notifica interfața Angular despre progresul procesării imaginilor. Implementarea practică constă într-un hub SignalR pe server care menține conexiunile WebSocket cu toți clienții activi. Funcția nativă C++ acceptă un pointer către o funcție callback (*ProgressCallback)* pe care o apelează periodic pe măsura avansării procesării imaginii. Acest callback este mapat, prin P/Invoke, la o funcție C# ce emite evenimente SignalR către client, transmițând procentul de progres actual. Astfel, utilizatorul care a încărcat o imagine vede în timp real, în interfața web, cum evoluează aplicarea filtrului. Această capabilitate real-time îmbunătățește experiența utilizatorului și demonstrează integrarea fluidă a mai multor tehnologii: cod nativ performant, API REST robust și actualizări interactive folosind WebSocket.

În rezumat, .NET 8 și limbajul C# au furnizat infrastructura necesară construirii unui serviciu web fiabil și eficient, capabil să orchestreze componentele externe (biblioteca C++ pentru calcul intensiv, baza de date, stocarea fișierelor și conexiunile cu clienții). Alegerea C# s-a datorat atât productivității ridicate oferite de limbaj (sintaxă concisă, gestionarea automată a memoriei prin garbage colector), cât și ecosistemului bogat de biblioteci disponibile pe .NET (de exemplu, driver pentru MongoDB, SDK pentru Dropbox, integrare SignalR etc.). Combinarea acestor facilități cu posibilitatea de a apela cod nativ a permis realizarea unui backend hibrid, îmbinând ușurința dezvoltării în C# cu performanța codului C++ nativ.

3.3. Angular 19 (Frontend web)

Partea de interfață a aplicației a fost dezvoltată folosind Angular, versiunea 19, un cadru de dezvoltare frontend modern pentru aplicații web de tip single-page. Angular este o platformă și un framework pentru constuirea de aplicații client single-page folosind HTML și TypeScript, dezvoltat și menținut de Google. Acesta oferă un ecosistem cuprinzător (prin unelte CLI, biblioteci și o structură bine definită) ce permite crearea de aplicații web dinamice și reactive. Angular extinde capabilitățile HTML prin sintaxă suplimentară (directive, data binding bidirecțional etc.), iar folosirea TypeScript – un superset al JavaScript-ului, tipizat static – îmbunătățește calitatea și mentenabilitatea codului frontend. Datorită sistemului de tipuri, multe erori pot fi depistate la compilare, înainte de rularea în browser, ceea ce conduce la o dezvoltare mai sigură și mai ușor de întreținut în aplicații de mari dimensiuni.

În cadrul proiectului, aplicația Angular constituie interfața grafică prin care utilizatorii interacționează cu sistemul de prelucrare a imaginilor. Aceasta permite încărcarea imaginilor sursă, selectarea filtrului dorit ți inițierea procesării, apoi afișează imaginea rezultată și informații despre aceasta. Comunicarea cu backend-ul .NET se realizează prin intermediul serviciilor web REST: Angular folosește modulul *HttpClient* pentru a trimite cereri HTTP (de tip POST, GET etc.) către endpoint-urile expuse de server. De exemplu, la trimiterea unei imagini spre procesare, frontend-ul efectuează un HTTP PUT conținând parametrii necesari (id-ul imaginii și filtrul ales), iar la finalizare primește de la API imaginea procesata. Aplicația este proiectată conform arhitecturii MVC (Model-View-Controller) specifică Angular: logica de interacțiune cu API-ul și gestionarea datelor este încapsulată în servicii și componente, separat de template-urile de afișare. Această separare face ca aplicația să fie usor extensibilă și testabilă, respectând totodată principiile de dezvoltare pe care Angular le promovează.

Un aspect remarcabil al integrării frontend/backend este folosirea SignalR și pe partea de client. Angular, prin intermediul bibliotecii *@aspnet/signalr* (clientul JavaScript pentru SignalR), se conectează la hub-ul SignalR al serverului imediat ce utilizatorul începe o operațiune de filtrare. Astfel, componenta responsabilă de afișarea progresului se abonează la evenimentele emise de server și actualizează în timp real starea procesării (de exemplu, procentul curent și un mesaj de notificare când procesarea s-a încheiat). Această abordare elimină necesitatea interogărilor repetate (polling) către

server, optimizând traficul și reducând latența percepută de utilizator. Din punct de vedere al experienței, utilizatorul vede aproape instantaneu evoluția sarcinii trimise, interfața reacționând fluid datorită modelului de programare reactivi al Angular (bazat pe RxJS) care permite actualizarea automată a componentelor UI la schimbarea datelor.

Folosirea Angular 19 asigură și alinierea la cele mai noi tehnologii frontend: motorul de randare Ivy, suport îmbunătățit pentru performanță și optimizări la build (compilare Ahead-of-Time, minimizare, împărțirea pe bundle-uri). Aplicația rezultată este responsive și compatibilă cu diferite browsere și dimensiuni de ecran, datorită atât framework-ului, cât și utilizării de tehnologii web standard (HTML5, CSS3). În concluzie, Angular a fost ales pentru că oferă un cadru solid de dezvoltare, unificând TypeScript pentru siguranța tipurilor, un sistem modular de componente și servicii, precum și o bună integrare cu un backend REST. Toate acestea au permis realizzarea unei interfețe web bogate, ușor de folosit de către utilizator și strâns legată de sistemul de procesare a imaginilor de pe server.

3.4. MongoDB

Sistemul de gestiune a bazelor de date folosit în acest proiect este MongoDB, o bază de date NoSQL de tip document. MongoDB stochează datele sub formă de documente flexibile, similare cu structurile JSON, ceea ce înseamnă că câmpurile pot varia de la un document la altul, iar structura datelor poate evolua în timp fără a necesita scheme rigide. Această flexibilitate a fost un motiv esențial pentru alegerea sa: în aplicația de procesare a imaginilor, era nevoie de stocarea unor metadate eterogene despre imagini (cum ar fi denumirea fișierului, tipul filtrului aplicat, etc.). Folosind MongoDB, s-a putut defini un document pentru fiecare imagine procesată, care conține aceste atribute, putând adăuga ușor câmpuri noi pe viitor fără migrări complexe ale unei scheme relaționale.

Un alt tip de informație stocat în MongoDB în cadrul proiectului îl reprezintă *cheile API* și datele de autentificare/autorizare. Aplicația expune un API public, iar pentru controlul accesului s-a implementat un mecanism de chei API unice pentru fiecare client. Aceste chei, împreună cu eventuale date despre utilizatori sau token-uri, sunt păstrate în colecții MongoDB, ceea ce permite o verificare rapidă a cererilor care ajung la server. Modelul de date oirentat pe document se potrivește bine și acestui scenariu – fiecare cheie API poate fi un document ce include și informații despre drepturile asociate sau

limita de utilizare. Interogările în MongoDB sunt eficiente datorită posibilității de indexare a câmpurilor chiar și în documente JSON, astfel încât căutarea după o cheie API sau după ID-ul unei imagini se realizează rapid.

Integrarea MongoDB cu platforma .NET s-a realizat cu ajutorul driver-ului oficial MongoDB pentru C#. Aceasta oferă o interfață object-oriented către baza de date, permițând efectuarea de operațiuni CRUD (Create, Read, Update, Delete) prin metode familiare (de exemplu, colecțiile MongoDB pot fi gestionate ca și cum ar fi liste de obiecte C#). În plus, s-au putut defini clase C# care mapează structura documentelor din baza de date, facilitând serializarea și deserializarea automată a datelor. Această abordare de tip ODM (Object-Document Mapping) reduce erorile și face codul mai lizibil, deoarece manipulează obiecte native în loc de documente brute. Deși aplicația nu a trebuit să gestioneze volume masive de date, MongoDB este recunoscut pentru scalabilitatea sa, permițând distribuirea datelor pe mai multe noduri și replicarea lor pentru disponibilitate ridicată. Astfel, alegerea MongoDB conferă și posibilitatea extinderii ulterioare a sistemului către un număr mai mare de utilizatori sau volum de informații, fără schimbarea tehnologiei de bază.

3.5. Dropbox (stocare fișiere)

Pentru stocarea fișierelor de imagine (atât a celor încărcate de utilizatori, cât și a rezultatelor obținute după procesare) s-a optat pentru integrarea cu serviciul Dropbox. Dropbox este o platformă de tip cloud storage bine cunoscută, care oferă și o interfață API pentru dezvoltatori. API-ul Dropbox permite aplicațiilor să lucreze cu fișierele stocate în conturile Dropbox ale utilizatorilor, incluzând funcționalități avansate precum căutare full-text, generarea de miniaturi sau partajarea fișierelor. În contextul acestui proiect, rolul Dropbox este de a externaliza stocarea conținutului multimedia, evitând încărcarea excesivă a serverului propriu cu fișiere mari și asigurând, totodată, redundanța și accesibilitatea datelor din orice locație.

Aplicativ, fluxul funcționează astfel: atunci când o imagine este încărcata de utilizator pentru a fi procesată, serverul o salvează temporar și apoi o încarcă în contul Dropbox asociat aplicației (folosind credențialele și token-ul de acces furnizate de platforma Dropbox). Operația se realizează printr-un apel HTTP la endpoint-urile Dropbox API, însoțit de datele fișierului, sau prin SDK-ul oficial Dropbox pentru .NET care simplifică aceste apeluri la nivel de cod C#. După procesare, imaginea rezultată este de

asemenea încărcată în Dropbox. Aplicația păstrează în baza de date MongoDB doar metadatele și link-urile necesare pentru a accesa aceste fișiere. De pildă, pentru fiecare imagine procesată se stochează un URL sharable generat de Dropbox și un identificator al fișierului; în momentul în care clientul solicită descărcarea rezultatului, API-ul nostru descarcă din Dropbox și retransmite datele brute.

Această soluție aduce multiple beneficii arhitecturale. În primul rând, separarea fișierelor de datele aplicației crește fiabilitatea – fișierele sunt replicate în cloud-ul Dropbox și nu se pierd în caz de cădere a serverului nostru. În al doilea rând, se facilitează scalarea: dacă volumul de imagini crește, se pot mări capacitățile contului Dropbox independent de infrastructura serverului web. Nu în ultimul rând, securitatea este luată în considerare – accesul la fișierele stocate se face prin conexiuni securizate HTTPS, iar token-urile de acces la Dropbox sunt protejate pe server (nefiind expuse în codul client). Per total, integrarea Dropbox a permis obținerea unui sistem de stocare fișiere robust și ușor de utilizat, fără a fi nevoie de implementarea unui server de stocare propriu de la zero.

3.6. Doxygen

Pentru documentarea automată a codului sursă al aplicației backend (în special a bibliotecii C++ de procesare a imaginilor), a fost utilizată unealta Doxygen. Doxygen este un generator de documentație larg utilizat în dezvoltarea software, care automatizează generarea documentației pornind de la comentariile din codul sursă. Practic, dezvoltatorul adnotează funcțiile, clasele și metodele cu comentarii speciale (folosind sintaxa Doxygen, de exemplu */** ... */* în C++ cu tag-uri precum *@brief*, *@param*, *@return*), apoi Doxygen analizează automat fișierele și produce documentația într-un format lizibil (HTML, PDF, LaTeX etc.).

În proiectul de față, Doxygen a fost cinfigurat să proceseze codul C++ din biblioteca ImagesProcessor. S-au adăugat comentarii detaliate pentru fiecare funcție. Aceste comentarii sunt apoi preluate de Doxygen pentru a genera o documentație API coerentă. Documentația rezultată include descrieri ale claselor, diagrama relațiilor între clase, lista de funcții și semnăturile lor, făcând astfel mult mai ușoară înțelegerea structurii modulului de procesare. De asemenea, Doxygen poate crea index de funcții, index de fișiere și grafice de dependență, oferind o vedere de ansamblu asupra codului.

Generarea automată a documentației cu Doxygen prezintă avantaje atât pe parcursul dezvoltării, cât și în etapa de livrare a proiectului. În timpul dezvoltării, programatorii se asigură că fiecare componentă este bine documentată, ceea ce facilitează mentenanța și extinderea ulterioară a codului (comentariile actualizate servesc ca documentație vie, sincronizată cu implementarea). Astfel, Doxygen a contribuit la calitatea proiectului, asigurând transparența codului și ușurința de utilizare a bibliotecii C++ de către alte părți interesate (de exemplu, daca alt dezvoltator ar dori să o folosească sau să o extindă).

3.7. Docker

Pentru a asigura portabilitatea și ușurința de desfășurare (deployment) a aplicației backend, s-a recurs la tehnologia de containerizare Docker. Docker este o platformă deschisă pentru dezvoltarea, expedierea și rularea aplicațiilor, care permite izolarea acestora de infrastructura subiacentă. Cu Docker, aplicația (inclusiv toate dependințele ei) este împachetată într-un container – un mediu de rulare standardizat – asigurând că software-ul va rula în același mod indiferent de mediul în care este lansat. În esență, containerizarea evită problemele de tipul „funcționează pe calculatorul meu, dar nu și pe server", oferind un mediu unificat de execuție.

În cazul proiectului de față, s-a creat o imagine Docker pentru aplicația .NET (backend) care include atât codul API-ului REST, cât și biblioteca nativă C++ compilată pentru sistemul de operare țintă. Procesul a fost automatizat folosind un fișier Dockerfile multi-stage: într-o primă etapă are loc compilarea aplicației (pas în care se generează și biblioteca *ImagesProcessor* corespunzătoare platformei vizate, de exemplu un *.so* pe Linux), iar în a doua etapă se pornește de la o imagine de runtime în care se copiază rezultatele build-ului. Astfel rezultă o imagine finală care conține aplicația ASP.NET 8, fișierul nativ (DLL/SO) și toate dependințele necesare (de ex. pachetele NuGet restaurate, librăriile de sistem etc.). Această imagine poate fi rulată atât pe Windows, cât și pe Linux x64, cu condiția să existe un engine Docker compatibil pe gazdă.

Containerizarea backend-ului aduce câteva beneficii cheie. În primul rând, facilitează deplin portabilitatea: dezvoltatorii pot rula local containerul, îl pot testa, apoi același container poate fi pus în producție, eliminând variaibilele legate de configurații de sistem diferite. În al doilea rând, Docker oferă izolarea aplicației – containerul are

propriul sistem de fișiere, propriile variabile de mediu și dependințe, deci nu interferează cu alte aplicații de pe aceeași mașină. În al treilea rând, folosirea containerelor deschide calea către orchestrare și scalare orizontală: dacă se dorește rularea mai multor instanțe ale serviciului de procesare imagini (pentru a gestiona un număr mare de cereri), instrumente precum Kubernetes pot fi folosite pentru a replica containerul pe multiple noduri. Deși proiectul de față a avut o scară limitată, pregătirea sa sub formă de container Docker face upgrade-urile și eventualele *deployment-uri* ulterioare mult mai simple (este suficientă obținerea unei noi imagini Docker și rularea ei, fără a reconfigura manual mediul). Nu în ultimul rând, containerul Docker al aplicației include și configurațiile necesare, acestea putând fi gestionate securizat prin mecanisme Docker/Kubernetes (ex. secrets). În concluzie, Docker a asigurat un mod eficient de ambalare și distribuire a aplicației backend, garantând că aceasta rulează uniform pe diverse platforme (Windows/Linux) și integrându-se în fluxul modern de livrare continuă.

3.8. Github Actions (CI/CD)

Pentru a menține calitatea codului și a automatiza procesul de construire și testare al aplicației, proiectul a folosit GitHub Actions ca soluție de integrare continuă și livrare continuă (CI/CD). Github Actions este o platformă CI/CD integrată în GitHub care permite automatizarea pipeline-ului de build, test și deploy al codului direct din repository. Practic, de fiecare dată când se efectuează modificări în cod (de exemplu, la fiecare push sau pull request în repository-ul de cod sursă), workflow-urile definite în YAML sunt declanșate pentru a rula diverse sarcini predefinite.

În cadrul acestui proiect, s-a configurat un pipeline CI care să acopere atât partea de backend (C++ și .NET), cât și partea de frontend (Angular). Astfel, au existat job-uri distincte în GitHub Actions pentru: Build și testare backend pe Windows, Build și testare backend pe Linux și Build și testare frontend. În job-urile de backend, workflow-ul instalează mai întâi mediul necesar (pentru Windows – Visual Studio build tools, pentru Linux – pachetul de .NET SDK și un compilator C++ GCC compatibil), apoi compilează biblioteca C++ (folosind CMake și generatorul specific platformei) și aplicația .NET 8. După un build cu succes se rulează testele unitare ale backend-ului: pe Windows, de exemplu, se rulează un executabil de teste C++ și testele .NET (folosind *dotnet test* pentru proiectele de test C#). Similar, pe Linux se execută analog teste pentru a asigura

că atât codul nativ cât și cel managed funcționează corect și pe această platformă. Job-ul pentru frontend presupune instalarea Node.js în runner-ul GitHub Actions, instalarea dependințelor npm și rularea testelor unitare cu Karma/Jasmine.

Un beneficiu major al folosirii GitHub Actions este verificarea constantă a compatibilității multiplatformă a codului. Dat fiind că biblioteca C++ țintește atât Windows cât și Linux, pipeline-ul definit include runneri pe ambele sisteme de operare, garantând că eventualele diferențe de compilator sau de mediu sunt detectate devreme. Spre exemplu, o modificare de cod care funcționează pe MSVC dar nu și pe gcc va fi semnalată de eșecul job-ului Linux, permițând dezvoltatorului să corecteze problema înainte de a livra software-ul. De asemenea, rularea automată a testelor la fiecare modificare asigură că nu se introduc regresii: dacă un test eșuează, acțiunea marchează build-ul ca defectuos, prevenind integrarea unei modificări problematice. Prin aceste mecanisme, GitHub Actions contribuie la menținerea calității și stabilității codului de-a lungul dezvoltării. În plus, platforma oferă posibilitatea de deploy continuu. Chiar dacă nu toate aceste etape au fost necesare în proiectul de față, existența pipeline-ului CI configurat a demonstrat aplicarea bunelor practici DevOps, sporind încrederea în produsul final.

3.9. Limbaje de programare utilizate: C++, C# și TypeScript

Proiectul a îmbinat trei limbaje de programare distincte, fiecare ales strategic pentru a acoperi o anumită latură a aplicației și pentru a valorifica atuurile respectivei tehnologii. În continuare sunt prezentate pe scurt aceste limbaje, împreună cu motivele selecției lor și beneficiile aduse în contextul de față.

C++ - este un limbaj de programare de nivel intermediar, cu performanțe de execuție foarte ridicate. Creat de Bjarne Stroustrup ca o extensie a limbajului C, C++ suportă programarea procedurală și pe obiecte, oferind programatorului control fin asupra managementului memoriei și resurselor sistem. În acest proiect, C++ a fost alegerea evidentă pentru implementarea algoritmilor intensivi de procesare a imaginilor: operațiile pe pixeli, filtrarea matricilor de imagine, transformările complexe necesitau eficiența la nivel de mașină pe care C++ o oferă. În plus, datorită compatibilității lui C++ cu C, a fost posibilă expunerea facilă a funcțiilor într-un mod

utilizabil de C# (folosind convenții C-style la interfața DLL-ului), făcând integrarea între componente mult mai lină.

C# - este un limbaj de programare modern, orientat pe obiecte, dezvoltat de Microsoft, conceput pentru a construi o gamă variată de aplicații ce rulează pe platforma .NET. C# combină sintaxa familiară din C/C++ cu caracteristici de siguranță și productivitate (gestionare automată a memoriei prin garbage collector, un sistem de tip static puternic, suport pentru programare asincronă, LINQ pentru interogarea colecțiilor, etc.). În cadul aplicației, C# a fost folosit pentru dezvoltarea API-ului web și a logicii de business asociate. Principalele motive ale alegerii C# au fost rapiditatea dezvoltării (framework-ul ASP.NET pune la dispoziție structuri gata făcute pentru controlere REST, routing, serializare JSON, autentificare ș.a.), precum și integrarea excelentă cu celelalte tehnologii server-side folosite: există drivere oficiale sau biblioteci ușor de utilizat în C# pentru MongoDB, pentru comunicarea cu Dropbox, pentru SignalR și altele, ceea ce a simplificat mult implementarea. De asemenea, C# fiind limbaj managed, multe erori comune (de exemplu, pointeri invalizi, buffer overflow) sunt evitate din start, crescând robustețea aplicației server. Nu în ultimul rând, platforma .NET asigură portabilitatea codului C# pe sisteme diferite, astfel că backend-ul scris în acest limbaj a putut rula fără modificări atât pe Windows cât și pe Linux (în mediul containerizat), îndeplinind cerințele multiplatformă.

TypeScript – este un superset al limbajului JavaScript, introdus de Microsoft, care adaugă tipare statice opționale și alte facilități menite să îmbunătățească scalabilitatea proiectelor web. Un motto definitoriu este că „TypeScript compilează în JavaScript simplu, orice cod JavaScript fiind și cod TypeScript valid, dar cu posibilitatea adăugării de anotări de tip”. Prin folosirea TypeScript în dezvoltarea frontend (impusă de fapt de framework-ul Angular, care este construit în jurul TypeScript), s-au obținut beneficii semnificative: detectarea timpurie a erorilor de tip (care altfel în JavaScript s-ar manifesta la runtime), abilitatea de a structura aplicația în clase și interfețe, oferind claritate arhitecturală. TypeScript a facilitat definirea de tipuri pentru datele transferate între frontend și backend (de exemplu, s-au putut crea interfețe TypeScript pentru răspunsurile JSON ale API-ului, descriind câmpurile așteptate, ceea ce ajută programatorul să folosească corect obiectele). Totodată, fiind un limbaj orientat obiect și funcțional în același timp, TypeScript a permis utilizarea unor pattern-uri de programare avansate în frontend, crescând calitatea codului client. Pe scurt, alegerea TypeScript a

fost motivată de nevoia unui frontend robust și ușor de întreținut, calități esențiale când interfața aplicației trebuie să gestioneze fluxuri complexe (încărcare de fișiere, afișarea progresului în timp real, actualizare dinamică a UI-ului pe baza mesajelor de la server etc.).

În concluzie, utilizarea combinată a C++, C# și TypeScript a permis exploatarea avantajelor fiecărui limbaj în parte în locul potrivit în arhitectura sistemului: C++ oferă viteză de execuție pentru calculul de pixeli, C# oferă un mediu de dezvoltare productiv și integrat pentru logica server și comunicarea cu alte servicii, iar TypeScript asigură un frontend fiabil și scalabil. Comunicarea între aceste limbaje s-a realizat prin mecanisme standard (P/Invoke pentru C++ ↔ C#, API REST + SignalR pentru C# ↔ TypeScript), rezultând un sistem unitar, deși eterogen în tehnologiile sale componente.

3.10. Medii de dezvoltare: Visual Studio și Visual Studio Code

Dezvoltarea proiectului a implicat utilizarea a două medii de dezvoltare integrate (IDE) principale, alese în funcție de specificul componentelor: *Visual Studio 2022* pentru partea de backend (atât pentru codul C++ cât și pentru API-ul .NET), respectiv *Visual Studio Code* pentru partea de frontend (aplicația Angular).

Microsoft Visual Studio este un mediu de dezvoltare integrat (IDE) foarte cuprinzător, furnizat de Microsoft, utilizat pentru a dezvolta programe de calculator, site-uri web, servicii web și aplicații mobile. Visual Studio oferă facilități avansate de depanare (debugging), lucru esențial mai ales pentru a diagnostica probleme la interfața dintre codul managed și unmanaged. Editorul puternic al Visual Studio, cu evidențiere sintactică, refactoring și navigare rapidă în cod, a accelerat scrierea codului complex (în special în C++, unde gestionează automat fișierele header și implementările, și poate genera definiții de metode ș.a.). Un alt avantaj a fost integrarea cu sistemul de control al versiunii GitHub: Visual Studio permite clonarea repository-ului, comiterea modificărilor și rezolvarea conflictelor direct din IDE.

Pentru dezvoltarea interfeței web în Angular, s-a preferat *Visual Studio Code*, un editor de cod sursă care a devenit foarte popular datorită flexibilității și vitezei sale. Visual Studio Code este un editor de cod gratuit (open-source) optimizat pentru construirea și depanarea aplicațiilor web și cloud moderne. Spre deosebire de Visual Studio, Visual Studio Code este mai ușor și extensibil prin plugin-uri. În cazul aplicației

Angular, Visual Studio Code a oferit o experiență excelentă datorită extensiilor dedicate: de exemplu, Angular Language Service a permis auto-completarea șabloanelor HTML și identificarea erorilor în template-uri, TSLint/ESLint a asigurat respectarea convențiilor de cod TypeScript, iar terminalul integrat a facilitat rularea rapidă a comenzilor Angular CLI (ng serve, ng test, ng build etc.) fără a părăsi editorul. Visual Studio Code dispune și de un debugger integrat pentru JavaScript/TypeScript – în proiect, a fost posibilă depanarea codului frontend direct din editor (folosind configurări de launch care atașează debugger-ul la Google Chrome, permițând puncte de întrerupere în codul TypeScript). Aceste capabilități au accelerat identificarea și remedierea eventualelor probleme din logica de interfață. Un alt motiv pragmatic pentru alegerea Visual Studio Code a fost faptul că proiectul frontend este bazat pe Node.JS și Angular CLI, unelte care nu necesită un IDE complex; Visual Studio Code excelează în a oferi exact uneltele necesare (highlight de sintaxă, IntelliSense pentru TypeScript, integrare Git) fără suprasolicitarea resurselor calculatorului, ceea ce a fost benefic în sesiunile lungi de dezvoltare.

În sinteză, folosirea celor două medii de dezvoltare a fost dictată de nevoile distincte ale componentelor proiectului. Visual Studio a asigurat un mediu bogat, enterprise-level, indispensabil pentru a coordona dezvoltarea simultană în C++ și C# și pentru a beneficia de depanare avansată. Pe de altă parte, Visual Studio Code a oferit agilitate și extensibilitate maximă pentru frontend, aliniindu-se cu fluxul tipic în dezvoltarea web modernă. Ambele unelte, complementare, au contribuit la creșterea productivității și calității codului scris.

Capitolul 4

4. Prezentarea aplicației: facilități și mod de utilizare

4.1. Descriere generală

Aplicația propusă reprezintă o soluție web modernă care permite utilizatorilor să încarce, să proceseze și să gestioneze imagini folosind o varietate de filtre și efecte, având la bază o arhitectură hibridă: backend performant în C++ și .NET 8, frontend modern în Angular 19, stocare în MongoDB/Dropbox și CI/CD folosind GitHub Actions. Toate aceste tehnologii asigură un flux rapid, stabil și scalabil, cu actualizări în timp real cătrea utilizator.

4.2. Facilități principale

Aplicația oferă o serie de facilități moderne care optimizează procesarea și gestionarea imaginilor digitale. Utilizatorii pot încărca rapid imagini fie prin drag & drop, fie folosind butonul dedicat de upload, având astfel o experiență intuitivă și accesibilă. O gamă variată de filtre și efecte – precum grayscale, invert, blur, sharpen, sobel, canny, sepia etc. – pot fi aplicate cu ușurință, rezultatele fiind afișate instant în interfață. Fiecare imagine procesată poate fi vizualizată și descărcată, iar aplicația păstrează un istoric arborescent al tuturor modificărilor, premițând urmărirea și compararea versiunilor derivate. Galeria oferă acces rapid atât la imaginile originale, cât și la cele procesate, fiecare fiind etichetată corespunzător. Actualizările privind progresul operațiunilor de procesare sunt transmise în timp real prin SignalR, oferind feedback constant utilizatorului. Pe partea de infrastructură, metadatele și managementul cheilor API sunt gestionate eficient în MongoDB, iar fișierele procesate sunt stocate securizat în Dropbox. Procesul de dezvoltare și mentenanțp este susținut de teste automate și pipeline-uri CI/CD, asigurând calitatea și rapiditatea livrărilor. De asemenea, aplicația este proiectată pentru a funcționa pe multiple platforme, atât pe Windows, cât și pe Linux, garantând astfel flexibilitate și portabilitate sporită.

4.3. Fluxul de utilizare

4.3.1. Pagina principală

Utilizatorul accesează pagina principală și poate încărca o imagine noua fie prin drag & drop, fie prin butonul de upload:

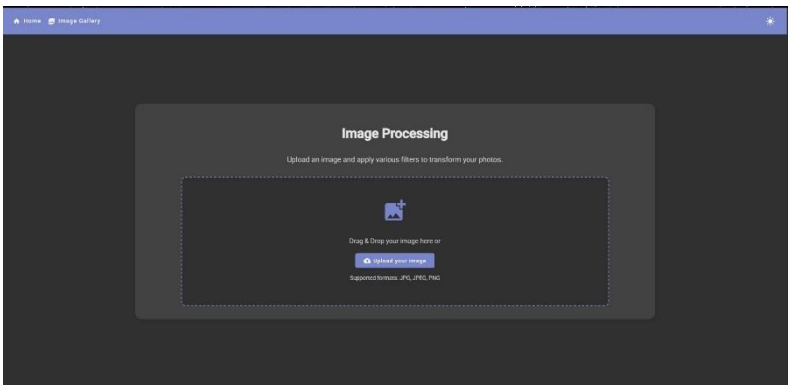

Fig. 4.1.: Pagina principală a aplicației

După selectarea imaginii, utilizatorul vede progresul încărcării:

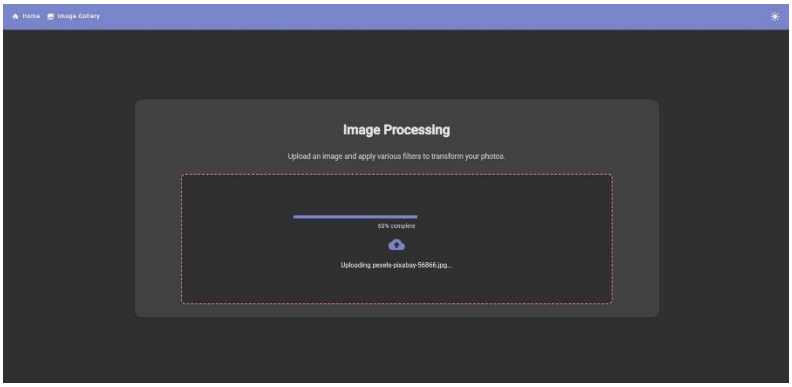

Fig. 4.2.: Progresul încărcării imaginii

4.3.2. Editarea imaginilor

După ce a fost încărcată imaginea, utilizatorul poate aplica diverse filtre disponibile în bara laterală:

Fig. 4.3.: Selectarea și aplicarea unui filtru

Pe durata procesării, progresul este afișat în timp real datorită integrării SignalR:

Fig. 4.4.: Progresul procesării cu SignalR

După finalizarea procesării, imaginea filtrată devine disponibilă pentru descărcare sau vizualizare:

Fig. 4.5.: Imaginea filtrată

4.3.3. Galerie de imagini

Galeria de imagini oferă utilizatorilor posibilitatea de a vizualiza rapid atât imaginile originale, cât și versiunile procesate, fiecare fiind clar marcată cu eticheta „Original" sau „Filtered". Acest lucru facilitează identificarea rapidă a stadiului fiecărei imagini. La plasarea cursorului (hover) peste o imagine, utilizatorul are acces la un set de opțiuni suplimentare: descărcarea rapidă a imaginii, inițierea editării sau vizualizarea structurii arborescente a versiunilor derivate din acea imagine. Printr-un simplu click pe o imagine, se deschide un dialog interactiv care permite compararea vizuală a pașilor de procesare și analiză detaliată a fiecărei etape de prelucrare.

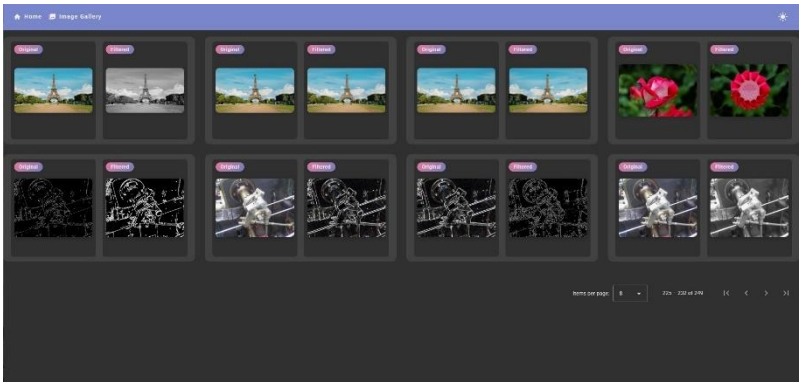

Fig. 4.6.: Galeria de imagini

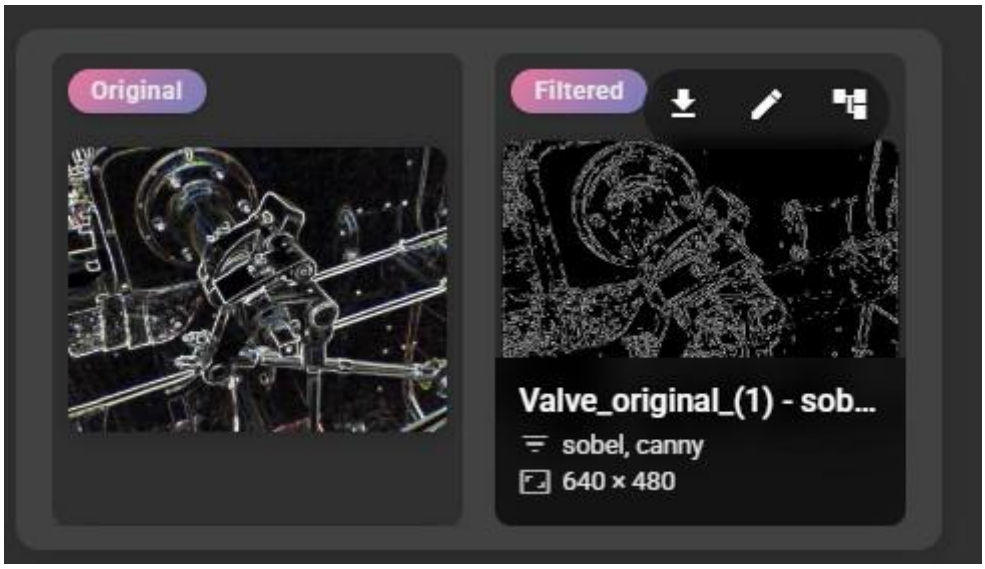

Fig. 4.7.: Detaliile despre imagine

4.3.4. Vizualizarea detaliată (Dialog)

Dialogul de vizualizare permite compararea vizuală a pașilor de procesare, oferind atât un view standard, cât și un view cronologic. Utilizatorii pot analiza cu ușurință diferențele dintre imaginea originală și versiunile rezultate după aplicarea fiecărui filtru.

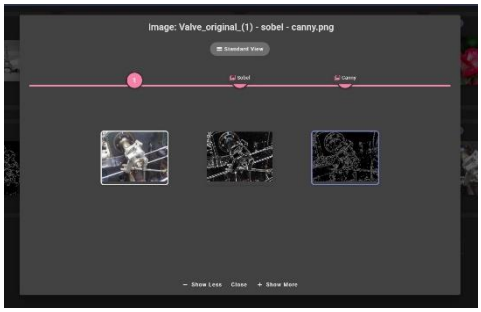

Fig. 4.8.: Dialogul cu vizualizarea cronologică

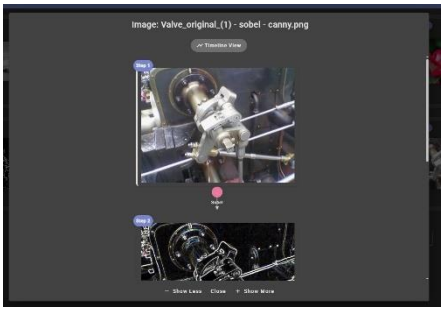

Fig. 4.9.: Dialogul cu vizualizarea standard

4.3.5. Vizualizarea arborescentă

Vizualizarea arborescentă permite utilizatorilor să observe cu ușurință istoricul complet al modificărilor pentru fiecare imagine, precum și relațiile dintre versiunile intermediare și filtrările aplicate succesiv.

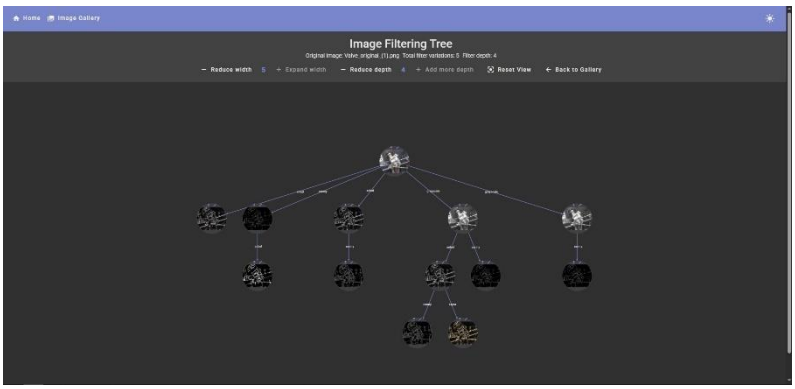

Fig. 4.10.: Structura arborescentă

Capitolul 5

5. Concluzii și rezultate

Proiectul a constat în dezvoltarea unei aplicații web moderne pentru procesarea imaginilor, care oferă utilizatorilor posibilitatea de a încărca și de a aplica o varietate de filtre și transformări în mod interactiv. Aplicația rezultată este una complexă, dar ușor de realizat, permițând prelucrarea rapidă a imaginilor și vizualizarea rezultatelor în timp aproape real. S-a realizat o interfață prietenoasă, bazată pe arhitectura *single-page application*, prin care utilizatorii pot selecta filtre, combina efecte și previzualiza modificările aduse fotografiilor lor. Mai multe filtre de bază au fost implementate (precum conversia unei imagini în tonuri de gri, invertirea culorilor, estomparea tip *blur*, accentuarea detaliilor *sharpen*, detectarea contururilor prin operatori Sobel sau Canny, ori efecte artistice de tip *oil paint*), demonstrând capacitatea sistemului de a suporta atât prelucrări clasice, cât și efecte vizuale avansate.

Fiecare operație de procesare este gestionată eficient, aplicația notificând utilizatorul cu privire la progresul procesării (în procente) și semnalând finalizarea fiecărei sarcini. Astfel, experiența de utilizare este una fluentă, fără întreruperi, iar utilizatorul are permanent feedback despre stadiul prelucrării imaginii sale.

Realizarea cu succes a proiectului a fost posibilă prin integrarea coerentă a unui set divers de tehnologii moderne, fiecare aducând beneficii specifice în arhitectura aplicației. Limbajul C++ a fost ales pentru implementarea algoritmilor de procesare a imaginilor, datorită eficienței sale ridicate și controlului oferit asupra memoriei și resurselor de calcul. Algoritmii de filtrare, implementați în C++, au permis prelucrarea rapidă a imaginilor chiar și la rezoluții mari, asigurând timpi de răspuns buni necesari unei aplicații interactive. Componenta de server a fost construită pe platforma .NET, mai precis folosind .NET CORE pentru a crea un API web robust și multiplatformă. Acest server .NET gestionează logica aplicației, primește imaginile încărcate de la clienți, apelează funcțiile native C++ pentru procesare și transmite înapoi rezultatele către client. Comunicarea în timp real între server și interfața web a fost realizată cu ajutorul tehnologiei SignalR, permițând actualizarea bidirecțională a informațiilor. Prin intermediul unui hub SignalR dedicat, serverul notifică imediat clientul web cu privire la

progresul procesării, îmbunătățind considerabil interacțiunea cu utilizatorul mai ales în cazul operațiilor mai lungi.

Pentru partea de interfață vizuală, s-a utilizat Angular, un framework modern de frontend care a permis dezvoltarea unei aplicații web reactive și ușor de întreținut. Alegerea Angular a facilitat crearea unei interfețe intuitive, în care utilizatorii pot administra imaginile și filtrele într-un mod interactiv, fără reîncărcarea paginii. Structura modulară oferită de Angular și sistemul de *data binding* au simplificat gestionarea stării aplicației și afișarea imediată a rezultatelor procesării în browser. Un alt aspect important este că interfața web realizată este una *responsive* (adaptivă), ceea ce înseamnă că aplicația este deja accesibilă și de pe dispozitive mobile.

Designul frontend-ului a fost conceput pentru a se adapta la ecrane de dimensiuni diferite, oferind o experiență consistentă atât pe monitoare de calculator, cât și pe ecranele mai mici ale telefoanelor mobile sau tabletelor. Acest lucru evidențiază caracterul multiplatformă al aplicației: datorită tehnologiilor web folosite, orice dispozitiv cu un navigator modern (indiferent de sistemul de operare) poate rula aplicația, fără a necesita modificări sau dezvoltări separate pentru fiecare platformă.

Persistența datelor și managementul informațiilor au fost asigurate prin integrarea unei baze de date NoSQL *MongoDB*. Aceasta a fost utilizată pentru stocarea în format document a datelor necesare aplicației. Abordarea bazată pe MongoDB a oferit flexibilitate în definirea schemelor de date și posibilitatea de a extinde ușor tipurile de informații stocate pe măsură ce aplicația evoluează. În plus, utilizarea unei baze de date NoSQL distribuibile se potrivește bine cu nevoia de scalabilitate a unei aplicații de acest tip, care ar putea gestiona un volum mare de imagini și operațiuni concurente.

Din perspectiva implementării și livrării soluției, s-au folosit instrumente moderne pentru asigurarea portabilității și a calității codului. În special, întregul sistem a fost containerizat cu Docker. Fiecare componentă (serverul .NET cu logica aplicației și componenta C++ a fost integrată într-un container Docker, asigurându-se astfel că aplicația poate rula în medii izolate și consistente. Docker a facilitat deplina multiplatformabilitate a aplicației la nivel de implementare: containerul poate fi desfășurat pe orice mașină (Windows, Linux sau macOS) sau în cloud, fără a necesita configurări specifice mediului respectiv. Acest mod de distribuție simplifică și procesul de instalare pentru utilizatorii finali sau pentru serverele de producție, deoarece toate dependințele și configurațiile sunt incluse în container. În paralel, pentru a menține un

ciclu de dezvoltare agil și fiabil, s-au folosit GitHub Actions ca soluție de integrare continuă și livrare continuă (CI/CD). Prin intermediul GitHub Actions a fost automatizată construirea aplicației și rularea testelor la fiecare modificare a codului sursă, asigurând detectarea timpurie a eventualelor probleme și menținând o versiune funcțională în permanență. De asemenea, pipeline-urile configurate au permis distribuirea rapidă a noilor versiuni. Aceste practici moderne de DevOps au contribuit la calitatea rezultată a proiectului.

Rezultatul final al proiectului îl reprezintă o aplicație de procesare a imaginilor complet funcțională, care îmbină performanța codului nativ C++ cu accesibilitatea soluțiilor web moderne. Toate obiectivele propuse inițial au fost atinse – de la implementarea principalelor filtre de imagine și realizarea unei interfețe web intuitive, până la asigurarea comunicării în timp real și a portabilității pe mai multe platforme. Soluția este scalabilă și extensibilă, putând fi cu relativă ușurință adaptată sau îmbunătățită pe viitor. În plus, caracterul multiplatformă și arhitectura bazată pe servicii web fac ca aplicația să poată servi drept fundație pentru dezvoltări viitoare în domeniul prelucrării imaginilor sau chiar al materialelor video. În continuare, vom prezenta câteva direcții posibile de extindere și optimizare a aplicației.

5.1. Direcții de extindere

Pe lângă funcționalitățile și performanțele deja obținute, există numeroase modalități prin care aplicația poate fi extinsă și perfecționată în viitor. Dintre direcțiile de dezvoltare posibile, se remarcă următoarele:

- *Suport pentru conținut video (încărcare și procesare):* O evoluție firească a platformei ar fi extinderea acesteia pentru a accepta și fișiere video, nu doar imagini statice. Implementarea suportului pentru upload de videoclipuri ar implica gestionarea fișierelor de dimensiuni mult mai mari și extragerea cadrelor (frame-urilor) relevante din acestea. Ulterior, aplicația ar putea aplica filtre pe fiecare cadru al unui clip video, realizând astfel filtrarea video. Deși procesarea video prezintă provocări suplimentare (datorită volumului mare de date și necesității de a menține sincronizarea temporală a cadrelor), extinderea în această direcție ar transforma aplicația într-o platformă completă de procesare multimedia. Această extindere ar necesita optimizări considerabile de performanță și posibil utilizarea accelerării hardware (GPU)

pentru a procesa cadrele video într-un timp rezonabil, însă ar crește substanțial utilitatea aplicației.

- *Optimizări de performanță ale algoritmilor existenți:* Chiar dacă aplicația funcționează deja eficient, există întotdeauna oportunități de a îmbunătăți performanța. Una dintre direcții ar fi optimizarea suplimentară a algoritmilor de procesare a imaginii. De exemplu, s-ar adopta tehnici de paralelizare mai agresivă, de exemplu prin *accelerare hardware* prin GPU/CUDA pentru operațiuni de filtrare intens computaționale. Astfel de optimizări ar reduce timpul de procesare per imagine, permițând prelucrarea mai rapidă a imaginilor de rezoluție foarte mare sau aplicarea simultană a mai multor filtre. De asemenea, s-ar putea îmbunătăți gestionarea memoriei și a resurselor, astfel încât aplicația să poată procesa un număr mai mare de imagini în paralel sau să ruleze eficient și pe dispozitive cu putere de calcul mai redusă. Orice creștere de performanță ar avea un impact direct asupra experienței utilizatorului, făcând interacțiunea și mai fluentă.
- *Extinderea gamei de filtre și funcții de procesare:* O altă direcție de extindere este adăugarea unor filtre noi și a unor funcționalități suplimentare de prelucrare a imaginilor. În forma actuală, aplicația oferă deja un set important de filtre clasice și efecte, însă domeniul prelucrării imaginilor este vast. Pe viitor, s-ar putea integra filtre avansate, precum filtre de corecție a culorilor, redimensionare inteligentă, eliminarea zgomotului de imagine prin algoritmi mai sofisticați sau chiar transformări geometrice complexe. De asemenea, un pas evolutiv ar fi includerea de algoritmi bazați pe inteligență artificială sau învățarea automată (machine learning), de pildă pentru îmbunătățirea calității imaginilor, detectarea și extragerea automată a obiectelor din imagini sau aplicarea de stiluri artistice folosind rețele neurale. Extinderea colecției de filtre ar menține interesul utilizatorilor și ar atrage noi cazuri de utilizare, consolidând aplicația ca un instrument versatil în prelucrarea imaginilor.
- *Îmbunătățiri ale interfeței și experienței de utilizare pe mobil:* Deși aplicația dispune deja de suport pentru dispozitive mobile prin interfața web responsivă, o direcție valoroasă ar fi dezvoltarea unei interfețe și mai optimizate pentru utilizarea pe smartphone-uri și tablete. Aceasta ar putea include rafinarea elementelor UI pentru ecrane tactile (butoane mai mari,

gesturi touch pentru zoom sau ajustarea intensității filtrului în timp real prin glisare etc.) și optimizarea performanței frontend astfel încât să consume mai puține resurse pe dispozitive mobile. O posibilitate ar fi chiar dezvoltarea unei aplicații mobile native complementare (pentru Android/iOS) sau utilizarea unui framework multiplatforma (precum Ionic, React Native sau .NET MAUI) care să reutilizeze componentele existente și să ofere o integrare mai strânsă cu sistemul de operare al dispozitivului (acces la cameră pentru a face poze direct din aplicație, notificări locale, partajare facilă către rețele sociale etc.).

- *Scalabilitate și distribuție în medii de producție:* O direcție tehnică importantă, mai ales dacă aplicația ar fi folosită de un număr mare de utilizatori, este asigurarea scalabilității și a disponibilității continue. În viitor, s-ar putea explora distribuirea componentelor aplicației în cloud, folosind orchestratori de containere precum Kubernetes pentru a gestiona mai multe instanțe ale serverului de procesare. De asemenea, s-ar putea implementa un sistem de balansare a sarcinii (load balancing) între mai multe servere, astfel încât procesarea imaginilor să fie împărțită și timpii de așteptare reduși chiar și sub un volum de cereri ridicat.

Implementarea acestor direcții de dezvoltare în viitor ar spori și mai mult capabilitățile și atractivitatea aplicației, transformând-o într-o platformă cu adevărat completă pentru procesarea imaginilor și chiar a conținutului video. Prin urmare, proiectul de față demonstrează că tehnologiile moderne (C++, .NET Core, Angular, SignalR, MongoDB, Docker, CI/CD etc.) pot conduce la realizarea unei aplicații multiplatformă robuste și performante. În concluzie, realizările proiectului confirmă viabilitatea soluției propuse, iar direcțiile de dezvoltare propuse indică potențialul de evoluție al aplicației.

Capitolul 6

6. Bibliografie

[1] Commons Wikimedia. RGB and CMYK comparison, 2024. URL: https://commons.wikimedia.org/wiki/File:RGB_and_CMYK_comparison.png

[2] Development Factory. Ce este un REST API, 2024. URL: https://developmentfactory.ro/blog/ce-este-un-rest-api

[3] Docs GitHub. Understanding GitHub Actions, 2024. URL: https://docs.github.com/en/actions/about-github-actions/understanding-github-actions

[4] Doxygen, 2024. URL: https://www.doxygen.nl/

[5] DSP Stack Exchange. Why is C so popular for image processing, 2024. URL: https://dsp.stackexchange.com/questions/25263/why-is-c-so-popular-for-image-processing

[6] Dynamsoft. Image Processing 101: Color Space Conversion, 2024. URL: https://www.dynamsoft.com/blog/insights/image-processing/image-processing-101-color-space-conversion

[7] GeeksforGeeks. HSV Color Model in Computer Graphics, 2024. URL: https://www.geeksforgeeks.org/hsv-color-model-in-computer-graphics/

[8] GeeksforGeeks. Image sharpening using Laplacian filter and high boost filtering in Matlab, 2024. URL: https://www.geeksforgeeks.org/image-sharpening-using-laplacian-filter-and-high-boost-filtering-in-matlab/

[9] GeeksforGeeks. Introduction to parallel programming with OpenMP in C++, 2024. URL: https://www.geeksforgeeks.org/introduction-to-parallel-programming-with-openmp-in-cpp/

[10] GeeksforGeeks. Program to change RGB color model to HSV color model, 2024. URL: https://www.geeksforgeeks.org/program-change-rgb-color-model-hsv-color-model/

[11] GeeksforGeeks. Sobel edge detection vs canny edge detection in computer vision, 2024. URL: https://www.geeksforgeeks.org/sobel-edge-detection-vs-canny-edge-detection-in-computer-vision/

[12] Homepages inf.ed.ac.uk. Mean filter, 2024. URL: https://homepages.inf.ed.ac.uk/rbf/HIPR2/mean.htm
[13] Medium. Sobel vs canny edge detection techniques: step by step implementation, 2024. URL: https://medium.com/@haidarlina4/sobel-vs-canny-edge-detection-techniques-step-by-step-implementation-11ae6103a56a
[14] Medium. What is C++, 2024. URL: https://medium.com/@ouzatl_88083/what-is-c-cb4a7e44ce59
[15] MongoDB. What is MongoDB, 2024. URL: https://www.mongodb.com/company/what-is-mongodb
[16] MuleSoft. Mule4 Dropbox Connector, 2024. URL: https://www.mulesoft.com/exchange/com.mulesoft.connectors/mule4-dropbox-connector/
[17] Prezi. Utilizarea filtrelor in Adobe Photoshop, 2024. URL: https://prezi.com/p/ptksm9cougcr/utilizarea-filtrelor-in-adobe-photoshop
[18] PubNub. SignalR Guide, 2024. URL: https://www.pubnub.com/guides/signalr/
[19] RapidTables. RGB to CMYK, 2024. URL: https://www.rapidtables.com/convert/color/rgb-to-cmyk.html
[20] RoleCatcher. Digital image processing, 2024. URL: https://rolecatcher.com/ro/skills/knowledge/arts-and-humanities/arts/digital-image-processing/
[21] Staticmania. Angular framework, 2024. URL: https://staticmania.com/blog/angular-framework
[22] Studocu. Referat grafică asistată de calculator, 2024. URL: https://www.studocu.com/ro/document/universitatea-politehnica-din-bucuresti/grafica-asistata-de-calculator/referat-grafica-asistata-de-calculator/20294820
[23] TheObjects. Sharpening filters, 2024. URL: https://www.theobjects.com/dragonfly/dfhelp/2024-1/Content/Image%20Filtering/Sharpening%20Filters.htm
[24] Tutorialspoint. Negative transformation of an image using Python and OpenCV, 2024. URL: https://www.tutorialspoint.com/negative-transformation-of-an-image-using-python-and-opencv

[25] Wikipedia Contributors. Canny edge detector, 2024. URL: https://en.wikipedia.org/wiki/Canny_edge_detector
[26] Wikipedia Contributors. Color space, 2024. URL: https://en.wikipedia.org/wiki/Color_space
[27] Wikipedia Contributors. Kernel (image processing), 2024. URL: https://en.wikipedia.org/wiki/Kernel_(image_processing)
[28] Wikipedia Contributors. Platform Invocation Services, 2024. URL: https://en.wikipedia.org/wiki/Platform_Invocation_Services
[29] Whitespace.md. Înțelegerea și ajustarea gamei de culori și a spațiului de culoare, 2024. URL: https://whitespace.md/intelegerea-si-ajustarea-gamei-de-culori-si-a-spatiului-de-culoare
[30] JDriven. Don't be afraid of Docker, 2020. URL: https://jdriven.com/blog/2020/05/dont-be-afraid-of-docker